

HW Week3: Day 9~11 Labs Report

김종원

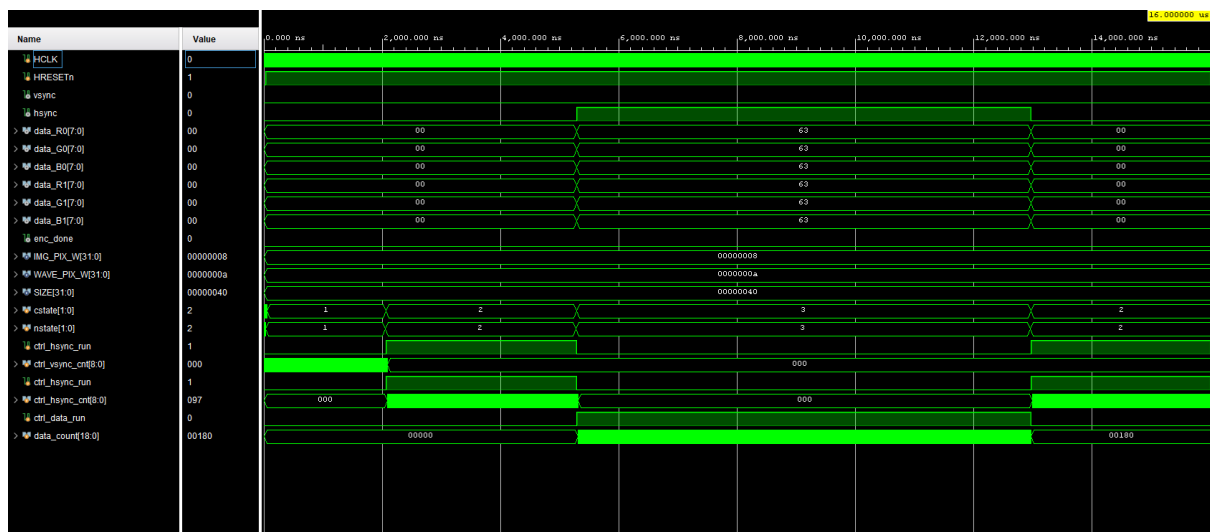
Day9. LAB1-(a,b) – LCD Drive System

본 실습에서는 Sensor Model과 Display Model을 이용한 LCD Drive System을 구현하였다. Sensor Model은 입력된 HEX 이미지 파일(kodim03.hex)을 읽어 RGB 이미지 데이터를 생성하고, LCD Controller의 동작을 모사하기 위해 VSYNC와 HSYNC 타이밍에 맞추어 두 개의 픽셀(R0, G0, B0 / R1, G1, B1)을 동시에 출력하도록 설계하였다. Display Model은 Sensor Model에서 출력되는 RGB 데이터를 수신하여 내부 메모리에 저장한 후, 모든 픽셀 수신이 완료되면 BMP 이미지 파일(kodim03.bmp)을 생성하도록 구현하였다.

실습에서는 sensor_model.v와 display_model.v의 Skeleton Code에서 제시된 Insert your code 부분을 완성하였다. Sensor Model에서는 입력 이미지를 읽어 RGB 데이터를 각각 org_R, org_G, org_B 배열에 저장하도록 구현하였으며, FSM(Finite State Machine)을 이용하여 IDLE → VSYNC → HSYNC → DATA 순으로 상태가 전이되도록 설계하였다. 또한 HSYNC Delay 이후 DATA 상태로 진입하여 두 개의 픽셀을 동시에 출력하도록 구현하였고, 모든 이미지 데이터의 전송이 완료되면 다시 IDLE 상태로 복귀하도록 구성하였다.

Display Model에서는 입력된 RGB 데이터를 BMP 포맷(BGR 순서)에 맞추어 내부 버퍼에 저장하였으며, 모든 데이터가 수신된 후 BMP Header와 Pixel Data를 순차적으로 파일에 기록하도록 구현하였다. 이를 통해 시뮬레이션만으로 출력 이미지를 생성하고 확인할 수 있도록 하였다.

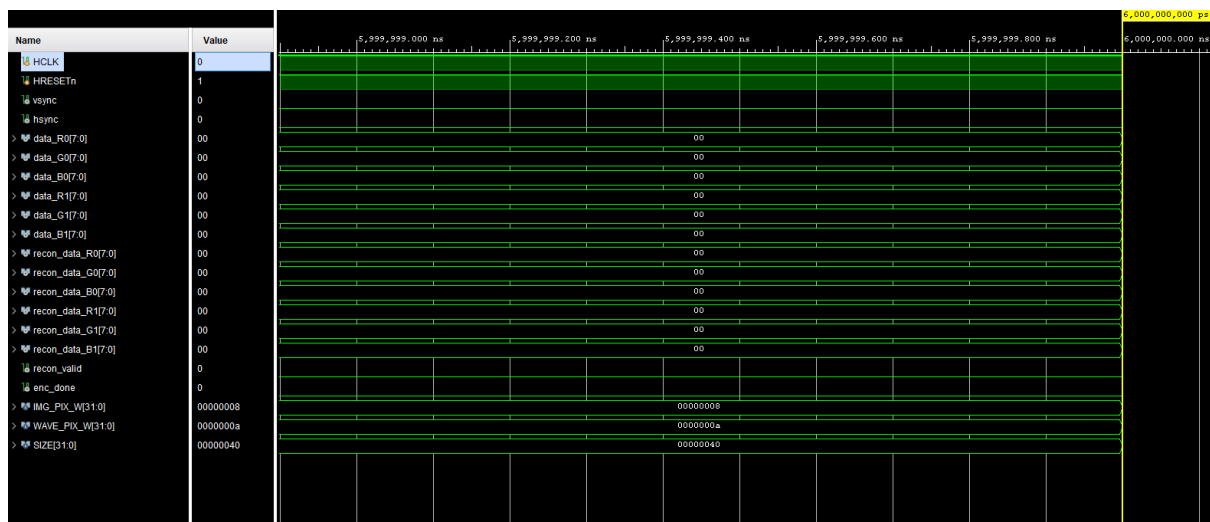
Lab 1-a에 대한 시뮬레이션 결과는 아래 그림과 같다.



첫 번째 Waveform을 보면 Reset이 해제된 이후 FSM이 VSYNC(State=1) → HSYNC(State=2) → DATA(State=3) 순으로 정상적으로 상태를 전이하는 것을 확인할 수 있다. VSYNC 구간에서는 ctrl_vsync_run이 활성화되어 프레임 시작을 위한 Delay가 수행되며, 이후 HSYNC 구간에서는 ctrl_hsync_run이 활성화되어 한 라인의 시작을 위한 Delay가 진행된다. HSYNC Delay가 종료되면 DATA 상태로 진입하여 ctrl_data_run이 활성화되고 RGB 이미지 데이터가 출력된다.

DATA 상태에서는 hsync 신호가 High 상태를 유지하며 DATA_R0, DATA_G0, DATA_B0와 DATA_R1, DATA_G1, DATA_B1을 통해 두 개의 픽셀 데이터가 동시에 출력되는 것을 확인할 수 있다. 또한 data_count가 DATA 상태에서 지속적으로 증가하는 것으로부터 이미지 데이터가 순차적으로 전송되고 있음을 확인하였다.

Lab 1-b에 대한 시뮬레이션 결과는 아래 그림과 같다.



Display Model Simulation에서는 약 6ms 동안 시뮬레이션을 수행한 후 모든 픽셀 데이터의 저장이 완료되면 DEC_DONE 신호가 발생하며, Display Model은 내부 메모리에 저장된 데이터를 이용하여 BMP 파일을 생성한다.

생성된 kodim03.bmp 파일을 확인한 결과 입력 이미지와 동일한 이미지가 정상적으로 출력되었다. 이미지의 색상 정보와 공간적인 구조가 원본과 동일하게 유지되었으며, RGB 데이터를 BMP 포맷(BGR 저장 방식)으로 변환하는 과정 또한 올바르게 수행되었음을 확인하였다. 이를 통해 Display Model의 데이터 저장 기능과 BMP 생성 기능이 모두 정상적으로 동작함을 확인할 수 있었다.

아래는 출력된 이미지이다.



즉, 실험 결과, 구현한 LCD Drive System은 Sensor Model이 입력된 HEX 이미지 데이터를 정상적으로 읽어 LCD 타이밍에 맞추어 RGB 데이터를 출력하고, Display Model이 이를 수신하여 BMP 이미지로 복원하는 기능을 성공적으로 수행하였다. Waveform을 통해 이미지 데이터가 LCD 인터페이스를 통해 정상적으로 전송되는 과정을 확인하였으며, Display Model에서 생성된 출력 이미지를 통해 RGB 데이터 저장 및 BMP 생성 기능이 올바르게 구현되었음을 검증하였다. 이를 통해 LCD Controller의 기본 동작 원리와 Sensor Model-Display Model 기반의 이미지 입출력 시스템을 이해할 수 있었다.

Day9. LAB2 – Brightness adjustment

본 실습에서는 Lab 1에서 구현한 LCD Drive System에 Brightness Adjustment Module을 추가하여 이미지의 밝기를 조절하는 시스템을 구현하였다. Sensor Model에서 출력되는 RGB 이미지 데이터를 Brightness Adjustment Module로 전달한 후, 설정된 Mode와 Value에 따라 각 픽셀의 밝기를 조정하고, 처리된 이미지를 Display Model로 전달하여 최종 BMP 이미지를 생성하도록 구성하였다.

Brightness Adjustment Module에서는 입력으로 두 개의 픽셀(R0, G0, B0 / R1, G1, B1)과 Valid 신호를 받아 Mode와 Value에 따라 출력 이미지를 생성하도록 설계하였다. Brighter Mode(Mode = 1)에서는 각 RGB 성분에 Value를 더하여 밝기를 증가시키고, 연산 결과가 255를 초과하는 경우에는 255로 고정(Saturation)하도록 구현하였다. 반대로 Darker Mode(Mode = 0)에서는 각 RGB 성분에서 Value를 감소시키고, 결과가 0보다 작아질 경우에는 0으로 고정하도록 구현하였다. 또한 입력 Valid 신호를 출력 Valid 신호로 전달하여 이미지 데이터의 타이밍이 유지되도록 설계하였다.

시뮬레이션 결과에 대한 사진은 아래 그림들과 같다.

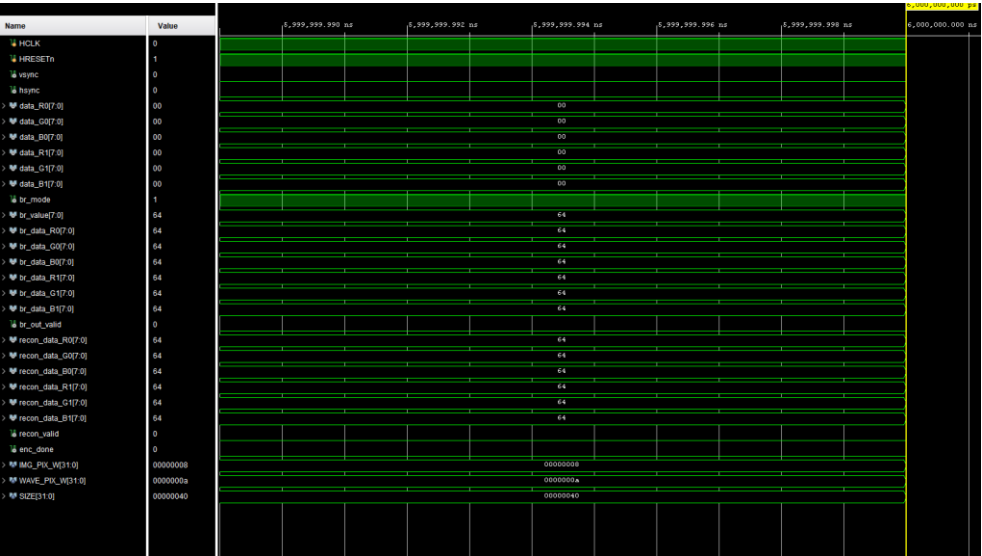
Baseline (bmp image)



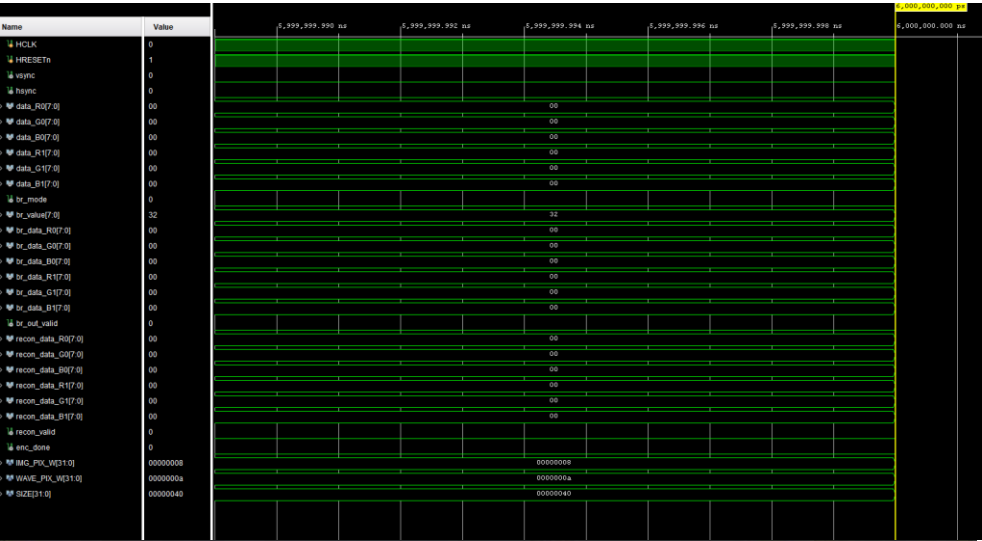
Brighter 50 (Waveform/bmp image)



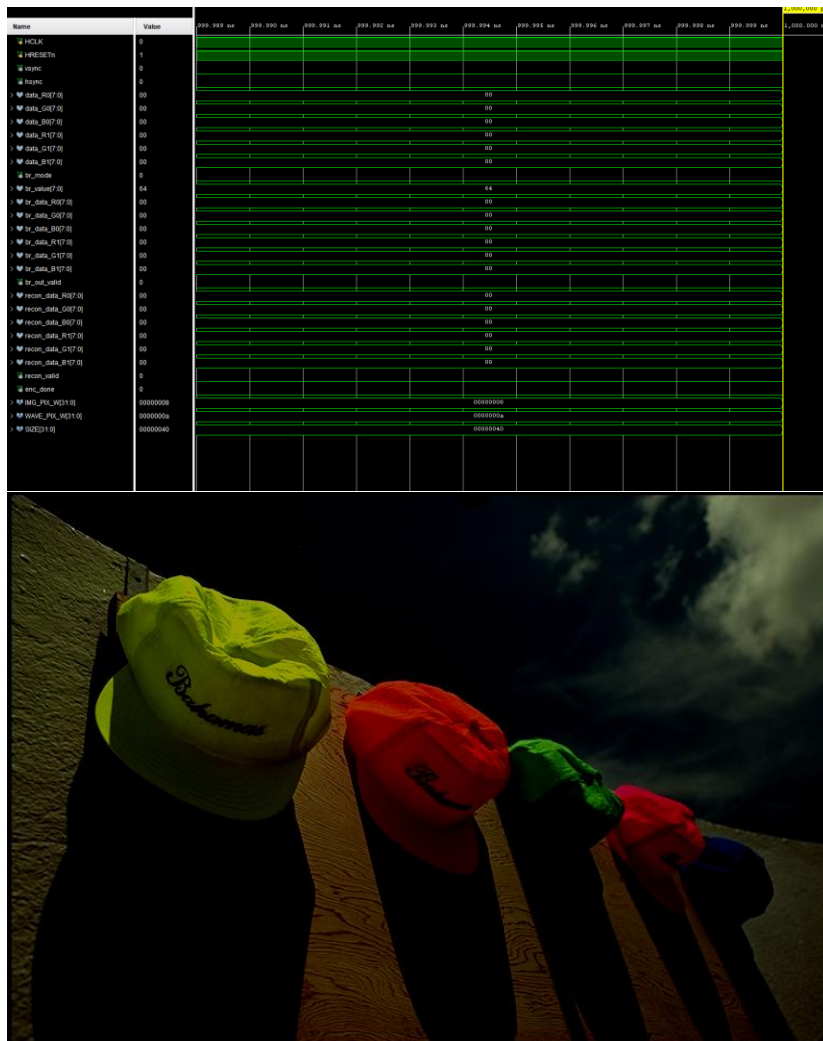
Brighter 100 (Waveform/bmp image)



Darker 50 (Waveform/bmp image)



Darker 100 (Waveform/bmp image)



Baseline은 Brightness Adjustment를 수행하지 않은 원본으로, Sensor Model에서 출력된 RGB 데이터가 그대로 Display Model에 전달되어 생성된 결과이다. 이를 기준으로 각 Brightness Mode의 동작을 비교하였다.

Brighter Mode에서는 Value를 각각 50과 100으로 설정하여 시뮬레이션을 수행하였다. Brighter 50에서는 전체 이미지의 밝기가 자연스럽게 증가하면서도 원본 이미지의 색상과 윤곽이 유지되는 것을 확인할 수 있었다. Brighter 100에서는 RGB 값이 크게 증가하여 이미지 전체가 Brighter 50보다 더욱 밝게 출력되는 것을 확인하였다. 이는 밝기 증가 연산이 정상적으로 수행되었음을 보여준다. 이를 통해 Brightness Adjustment Module의 Saturation 연산이 정상적으로 수행됨을 확인할 수 있었다.

Darker Mode에서는 Value를 각각 50과 100으로 설정하여 실험을 수행하였다. Darker 50에서는 전체적인 밝기가 감소하면서 이미지의 형태와 색상은 유지되었으나 명암이 더욱 짙어지는 결과를 확인하였다. Darker 100에서는 대부분의 RGB 값이 크게 감소하여 이

미지가 더욱 어두워진 것을 확인하였다.

실험 결과, 구현한 Brightness Adjustment Module은 입력된 RGB 이미지 데이터에 대해 Mode와 Value에 따라 이미지의 밝기를 정상적으로 조절하였으며, Display Model을 통해 처리된 이미지를 BMP 이미지로 복원할 수 있음을 확인하였다. 또한 Brighter Mode와 Darker Mode에서 Value를 변경함에 따라 출력 이미지의 밝기가 단계적으로 변화하는 것을 확인하여, 픽셀 단위의 RGB 연산이 정상적으로 수행됨을 검증하였다. 이를 통해 LCD Drive System에 이미지 처리 모듈을 추가하여 실시간으로 이미지를 처리하는 방법을 이해할 수 있었으며, Sensor Model, Brightness Adjustment Module, Display Model을 연동한 이미지 처리 파이프라인의 동작 원리와 데이터 흐름을 이해할 수 있었다.

Day9. LAB3 – Reversible Color Transform (RCT)

본 실습에서는 Reversible Color Transform(RCT)을 이용하여 RGB 이미지를 YCbCr 색 공간으로 변환한 후, 다시 RGB 이미지로 복원하는 LCD Drive System을 구현하였다. RCT는 이미지 압축에서 널리 사용되는 무손실 색 공간 변환 기법으로, RGB 채널 간의 높은 상관관계를 이용하여 휘도(Y)와 색 차(Cb, Cr) 성분으로 변환함으로써 이미지 데이터를 효율적으로 표현할 수 있다.

실습에서는 기존 Sensor Model과 Display Model을 그대로 재사용하고, 그 사이에 Forward RCT Module과 Inverse RCT Module을 추가하여 전체 이미지 처리 시스템을 구성하였다. Sensor Model은 입력된 kodim03.hex 파일을 읽어 RGB 데이터를 출력하며, Forward RCT Module은 이를 입력받아 YCbCr 데이터로 변환한다. 이후 Inverse RCT Module은 변환된 YCbCr 데이터를 다시 RGB 데이터로 복원하고, Display Model은 복원된 데이터를 BMP 이미지로 저장하여 최종 결과를 확인할 수 있도록 구성하였다.

강의 자료에 따르면, Forward RCT Module에서는 입력된 두 개의 픽셀 중 첫 번째 픽셀에 대해 색차 성분(Cb, Cr)을 계산하고 이를 두 픽셀이 공유하도록 구현되었다. 휘도(Y)는

$$Y = \frac{R + 2G + B}{4}$$

식을 이용하고, 하위 비트는 Truncation을 통해 제거한다. 또한 색차 성분은

$$Cb = B - G, Cr = R - G$$

식을 이용하여 계산한다. 이를 통해 두 개의 RGB 픽셀(48 bit)을 Y0, Y1, Cb, Cr의 네 개 값(40 bit)으로 변환하여 데이터 크기를 감소시킬 수 있도록 구현한다. Inverse RCT Module에서는 입력된 YCbCr 데이터를 이용하여 RGB 데이터를 복원한다. 먼저

$$tmp = Y - \frac{Cb + Cr}{4}$$

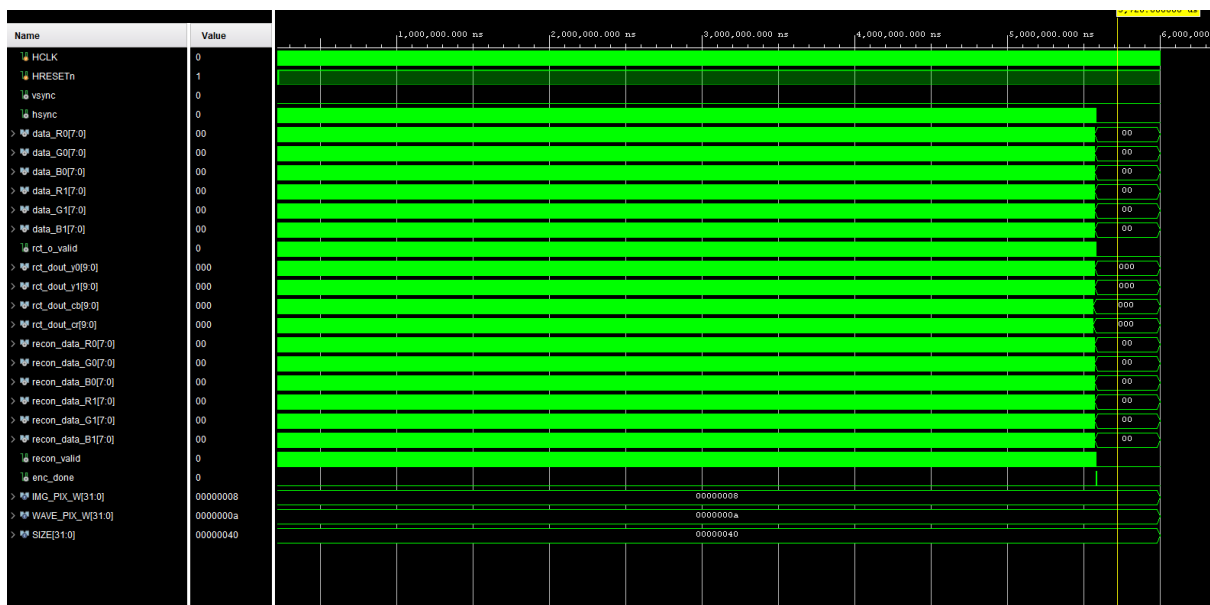
를 계산한 후,

$$R = Cr + tmp, G = tmp, B = Cb + tmp$$

공식을 이용하여 RGB 값을 복원한다. 또한 복원된 결과가 이미지의 표현 범위를 벗어나지 않도록 Clipping 기능을 적용하여, 255보다 큰 값은 255로, 0보다 작은 값은 0으로 제한하였다. 이를 통해 Overflow와 Underflow를 방지할 수 있도록 구현한다.

시뮬레이션 결과는 아래 그림과 같다.

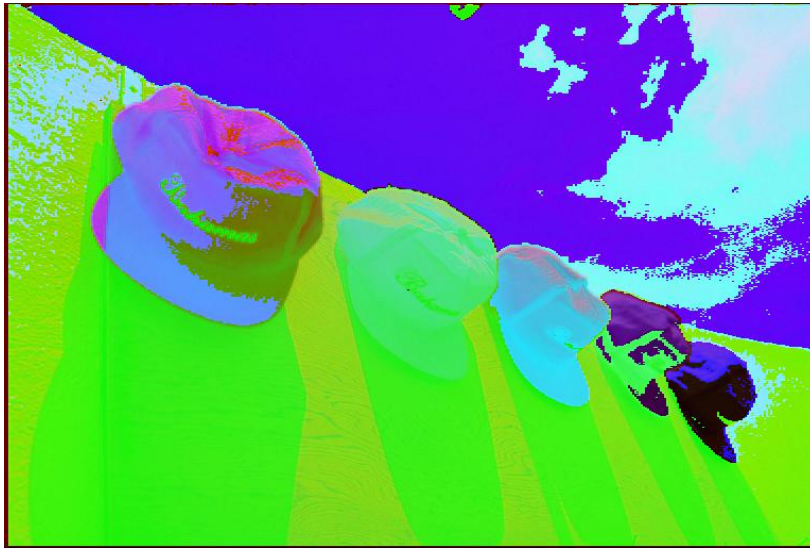
waveform



kodim03.bmp



kodim03_ycbcr.bmp



Forward RCT를 수행한 결과 생성된 kodim03_ycbcr.bmp를 확인하면, 원본 RGB 이미지와는 다른 형태의 이미지가 출력되는 것을 확인할 수 있다. 이는 RGB 이미지를 휘도(Y)와 색차(Cb, Cr) 성분으로 변환하여 저장하였기 때문으로, 화면의 밝기 정보는 비교적 유지되는 반면 색상 정보는 원본 이미지와 다른 형태로 표현된다. 또한 두 개의 픽셀이 하나의 Cb, Cr 값을 공유하도록 구현되어 색차 성분이 축소된 형태로 저장되는 것을 확인할 수 있었다.

Inverse RCT를 수행하여 생성된 kodim03.bmp를 확인한 결과, 입력 이미지와 거의 동일한 이미지가 복원되었다. 이미지의 전체적인 색상, 윤곽, 그리고 객체의 형태가 정상적으로 유지되었으며, 원본 이미지와 비교하여 눈에 띄는 왜곡이나 손실은 발생하지 않았다. 이를 통해 Forward RCT에서 생성된 YCbCr 데이터를 이용하여 RGB 이미지가 정상적으로 복원됨을 확인할 수 있었다.

실험 결과, 구현한 RCT 기반 LCD Drive System은 RGB 이미지를 YCbCr 색 공간으로 정상적으로 변환한 후 다시 RGB 이미지로 복원하는 기능을 성공적으로 수행하였다. 특히 출력 이미지를 통해 복원된 이미지가 원본과 거의 동일하게 유지되는 것을 확인함으로써 Forward RCT와 Inverse RCT가 서로 올바르게 동작함을 검증할 수 있었다. 또한 이번 실습을 통해 RGB와 YCbCr 색 공간의 관계를 이해하고, 색 공간 변환을 이용하여 이미지 데이터를 효율적으로 표현하면서도 원본 이미지를 복원할 수 있는 RCT의 동작 원리를 확인할 수 있었다.

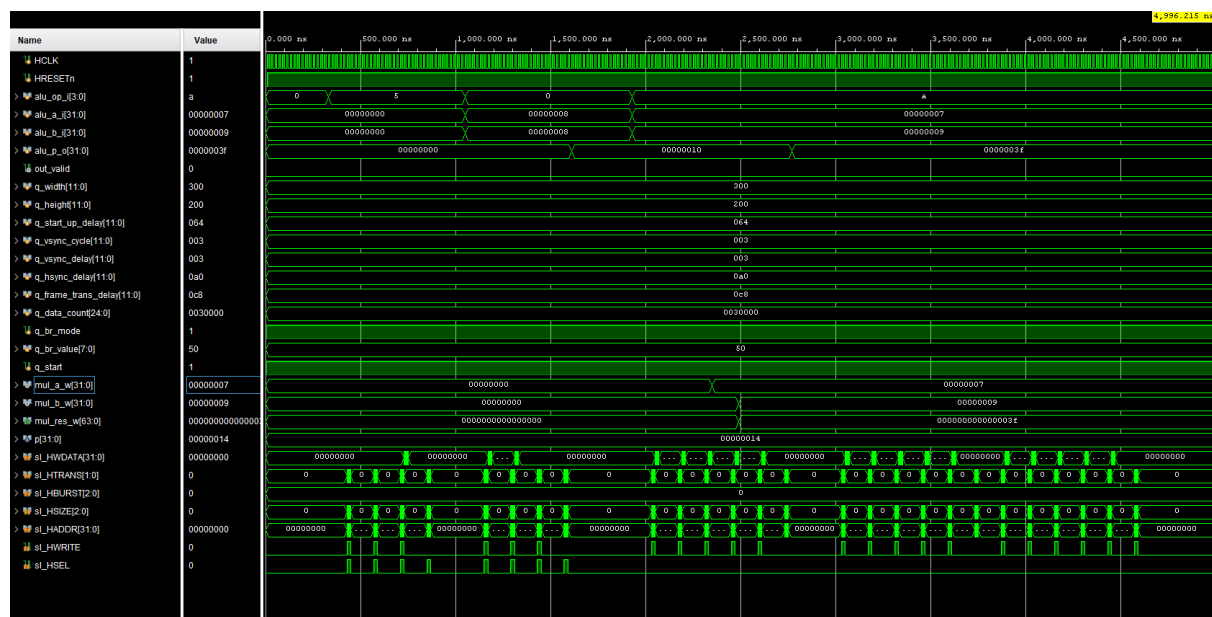
Day10. LAB1 – LCD drive AHB interface

(a) LCD Drive AHB Interface 구현

본 실습에서는 LCD Drive를 AHB Bus에 연결하기 위해 AHB Slave Interface (lcd_drive_if.v)를 구현하고, 이를 기존 SoC 시스템에 통합하였다. 기존 시스템은 AHB Master(RISC-V Dummy)와 ALU, Multiplier 두 개의 Slave IP로 구성되어 있었으며, 본 실습에서는 LCD Drive를 세 번째 AHB Slave로 추가하였다. 이를 위해 LCD Drive의 Base Address와 Register Map을 map.v에 정의하고, top_system.v의 AHB Interconnect에 LCD Drive IP를 연결하여 Master가 AHB Bus를 통해 LCD Drive를 제어할 수 있도록 설계하였다.

LCD Drive Interface는 AHB Slave Interface를 기반으로 구현되었으며, Address Phase에서 전달되는 HADDR을 이용하여 접근할 Register를 선택하고, HWRITE 신호를 저장하여 Read/Write Transaction을 구분하도록 Address Decode Logic을 구성하였다. Data Phase에서는 선택된 Register에 대해 Width, Height, Start-up Delay, VSYNC Cycle, VSYNC Delay, HSYNC Delay, Frame Transfer Delay, Data Count, Brightness Mode, Brightness Value 및 Start Register를 읽거나 쓰도록 구현하였다. 또한 Register에 저장된 설정값은 이후 LCD Drive의 동작을 제어하는 내부 제어 신호로 사용되도록 설계하였다.

시뮬레이션 결과는 아래 그림과 같다.



Waveform을 보면 AHB Master가 Bus Interconnect를 통해 LCD Drive Slave를 선택한 후, Address Phase와 Data Phase를 거쳐 각 제어 Register에 순차적으로 접근하는 것을

확인할 수 있다. AHB Interconnect는 전달된 주소를 디코딩하여 ALU, Multiplier 또는 LCD Drive 중 해당 Slave를 선택하며, LCD Drive의 주소 영역에 접근하는 경우 lcd_drive_if로 Transaction을 전달한다. 이후 LCD Drive Interface는 전달받은 주소를 Register Offset으로 변환하여 해당 Register를 선택하고, Write Transaction에서는 입력 데이터를 내부 Register에 저장하며, Read Transaction에서는 저장된 Register 값을 HRDATA를 통해 Master로 반환한다. 이를 통해 LCD Drive가 기존 SoC 시스템의 하나의 AHB Slave IP로 정상적으로 동작하며, Bus 기반으로 모든 동작 파라미터를 제어할 수 있음을 확인하였다.

(b) LCD Drive Data Signal 생성

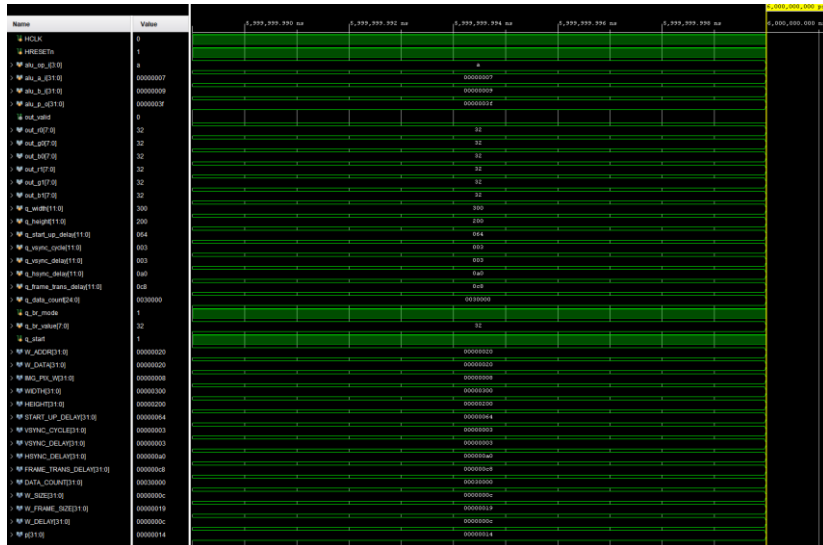
본 실습에서는 LCD Drive IP 내부의 FSM(Finite State Machine)을 완성하여 Display Panel에 출력하기 위한 데이터 생성 기능을 구현하였다. FSM은 AHB Bus를 통해 전달받은 화면 설정값(Width, Height, Delay 등)을 이용하여 Row Counter와 Column Counter를 제어하며, 화면의 각 픽셀 위치에 대응하는 RGB 데이터를 순차적으로 생성하도록 설계하였다. 또한 한 프레임의 모든 픽셀 출력이 완료되면 End-of-Frame 신호를 발생시켜 다음 프레임으로 전환하도록 구현하였다.

시뮬레이션은 6 ms 동안 수행하였으며, LCD Drive의 레지스터에 저장된 설정값을 기반으로 정상적으로 이미지가 출력되는 것을 확인하였다. Waveform을 보면 AHB Master가 먼저 LCD Drive의 Register에 화면 크기, 동기 신호 지연시간 및 데이터 개수 등의 초기 설정값을 기록한 후, Start Register를 통해 출력을 시작한다. 이후 LCD Drive 내부 FSM이 활성화되어 Row Counter와 Column Counter를 증가시키며 화면 전체를 순차적으로 스캔하고, 이에 따라 Display Model로 RGB 데이터와 Valid 신호가 지속적으로 전달되는 것을 확인하였다.

또한 Brightness Mode와 Brightness Value를 변경하여 여러 조건에서 실험을 수행하였다. q_br_mode 및 q_br_value를 변경함으로써 출력되는 RGB 값이 일정한 비율로 증가하거나 감소하여 화면의 밝기가 변경되는 것을 확인하였다. 생성된 출력 이미지를 비교한 결과, 프레임 구조나 이미지의 형태는 동일하게 유지되면서 밝기만 정상적으로 조절되는 것을 확인하였다.

시뮬레이션 결과는 아래 그림과 같다.

- 이미지의 순서대로, Brighter 50, 100, 150, Darker 50, 100, 150에 대한 Waveform이다.



[illegible]

- 이미지의 순서대로, Brighter 50, 100, 150, Darker 50, 100, 150 이다.



Waveform에서도 LCD Drive의 내부 Register 값이 정상적으로 유지되며, Width(300), Height(200), Start-up Delay, VSYNC/HSYNC Delay, Frame Transition Delay, Data Count 등의 설정 값이 FSM에서 사용되는 것을 확인하였다. 이후 q_start가 활성화되면서 프레임 출력이 시작되고, 내부 주소(W_ADDR), Frame Size(W_FRAME_SIZE) 및 출력 데이터가 연속적으로 갱신되어 Display Panel에 이미지를 전송하는 과정이 정상적으로 수행됨을 확인하였다.

실험 결과, LCD Drive를 AHB Slave IP로 구현하여 기존 SoC 시스템에 성공적으로 통합하였으며, AHB Master가 Bus Interconnect를 통해 LCD Drive의 Register를 정상적으로 제어할 수 있음을 확인하였다. 또한 LCD Drive 내부의 FSM과 Row/Column Counter를 이용하여 LCD Panel 구동에 필요한 제어 신호와 이미지 데이터를 생성하고, Brightness

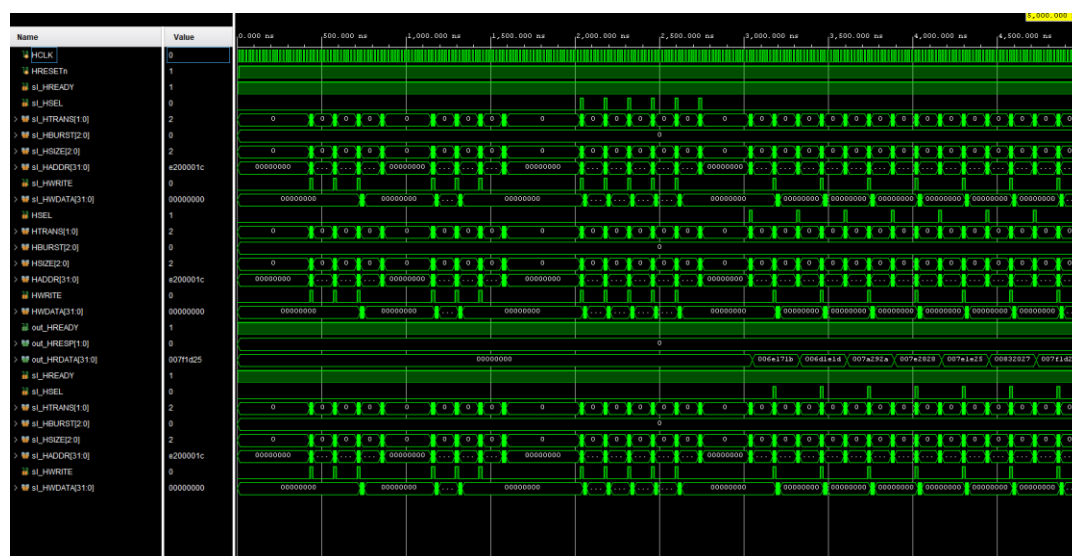
Adjustment 기능을 적용하여 밝기에 따른 이미지가 정상적으로 출력됨을 검증하였다. 이를 통해 AHB 기반 SoC에서 새로운 IP를 Bus Interconnect에 추가하고, Register 기반의 제어를 통해 LCD Drive를 구동하는 전체 동작 과정이 정상적으로 수행됨을 확인하였다.

Day10. LAB2 – BRAM Controller

본 실습에서는 LCD Drive를 AHB Bus에 연결하기 위해 AHB Slave Interface (lcd_drive_if.v)를 구현하고, 이를 기존 SoC 시스템에 통합하였다. 기존 시스템은 AHB Master(RISC-V Dummy)와 ALU, Multiplier 두 개의 Slave IP로 구성되어 있었으며, 본 실습에서는 LCD Drive를 세 번째 AHB Slave로 추가하였다. 이를 위해 LCD Drive의 Base Address와 Register Map을 map.v에 정의하고, top_system.v의 AHB Interconnect에 LCD Drive IP를 연결하여 Master가 AHB Bus를 통해 LCD Drive를 제어할 수 있도록 설계하였다.

LCD Drive Interface는 AHB Slave Interface를 기반으로 구현되었으며, Address Phase에서는 전달되는 HADDR를 이용하여 접근할 Register를 선택하고 HWRITE 신호를 저장하여 Read/Write Transaction을 구분하도록 Address Decode Logic을 구성하였다. Data Phase에서는 선택된 Register에 대해 Width, Height, Start-up Delay, VSYNC Cycle, VSYNC Delay, HSYNC Delay, Frame Transfer Delay, Data Count, Brightness Mode, Brightness Value 및 Start Register를 읽거나 쓰도록 구현하였다. 또한 Register에 저장된 설정 값은 LCD Drive 내부 FSM의 동작 파라미터로 사용되어 이미지 출력 과정에서 화면 크기와 동기 신호 생성, 밝기 조절 기능을 제어하도록 설계하였다.

시뮬레이션 결과는 아래 그림과 같다.



Waveform을 보면 AHB Master가 Bus Interconnect를 통해 LCD Drive Slave를 선택한

후 Address Phase와 Data Phase를 거쳐 각 제어 Register에 순차적으로 접근하는 것을 확인할 수 있다. AHB Interconnect는 전달된 주소를 디코딩하여 ALU, Multiplier 또는 LCD Drive 중 해당 Slave를 선택하며, LCD Drive 주소 영역에 접근하는 경우 lcd_drive_if로 Transaction을 전달한다. 이후 LCD Drive Interface는 전달받은 주소를 Register Offset으로 변환하여 해당 Register를 선택하고, Write Transaction에서는 입력 데이터를 내부 Register에 저장하며, Read Transaction에서는 저장된 Register 값을 HRDATA를 통해 Master로 반환한다. 또한 HREADY는 정상적으로 High 상태를 유지하고 HRESP는 OKAY 응답을 반환하여 오류 없이 Bus Transaction이 수행되는 것을 확인하였다. 이를 통해 LCD Drive가 기존 SoC 시스템의 하나의 AHB Slave IP로 정상적으로 동작하며, Bus 기반으로 모든 제어 파라미터를 설정할 수 있음을 확인하였다.

또한, 그 다음으로 LCD Drive IP 내부의 FSM(Finite State Machine)을 완성하여 Display Panel에 출력하기 위한 데이터 생성 기능을 구현하였다. FSM은 AHB Bus를 통해 전달받은 Width, Height, Delay 등의 설정 값을 이용하여 LCD 출력 타이밍을 제어하도록 설계되었다. DATA 상태에서는 Row Counter와 Column Counter를 이용하여 화면의 각 픽셀 위치를 순차적으로 접근하며, Frame Buffer에서 두 개의 Pixel(RGB 데이터)을 동시에 읽어 Display Model로 전달하도록 구현하였다. 또한 Data Counter를 이용하여 출력된 Pixel Pair의 개수를 계산하고, 모든 픽셀 출력이 완료되면 End-of-Frame 신호를 발생시켜 다음 프레임으로 전환하도록 구현하였다.

시뮬레이션은 20ms 동안 수행하였으며, LCD Drive Register에 저장된 설정 값을 기반으로 정상적으로 이미지가 출력되는 것을 확인하였다. Waveform을 보면 AHB Master가 먼저 LCD Drive Register에 화면 크기, 동기 신호 지연시간 및 데이터 개수 등의 초기 설정 값을 기록한 후, Start Register를 활성화하여 이미지 출력을 시작한다. 이후 LCD Drive 내부 FSM이 IDLE → VSYNC → HSYNC → DATA 순으로 상태를 전이하며, Row Counter와 Column Counter를 증가시켜 화면 전체를 순차적으로 스캔하고 Display Model로 RGB 데이터와 Valid 신호를 지속적으로 전달하는 것을 확인하였다.

q_br_mode와 q_br_value를 변경함으로써 출력되는 RGB 값이 일정한 값만큼 증가하거나 감소하여 화면의 밝기가 변경되는 것을 확인하였다. 생성된 출력 이미지를 비교한 결과, 프레임 구조와 이미지의 형태는 그대로 유지되면서 밝기만 정상적으로 조절되는 것을 확인하였다.

시뮬레이션 결과는 아래 그림들과 같다. (이미지의 순서대로 원본, Brighter 50, Brighter 100, Darker 50, Darker 100에 대한 Waveform 및 결과 bmp 파일이다.)

원본



Brighter 50

Name	Value	0x_177_000_000_mn	0x_180_000_000_mn	0x_180_000_000_mn	0x_180_000_000_mn	0x_180_000_000_mn	0x_180_000_000_mn
0	0						
1	1						
2	2						
3	3						
4	4						
5	5						
6	6						
7	7						
8	8						
9	9						
10	10						
11	11						
12	12						
13	13						
14	14						
15	15						
16	16						
17	17						
18	18						
19	19						
20	20						
21	21						
22	22						
23	23						
24	24						
25	25						
26	26						
27	27						
28	28						
29	29						
30	30						
31	31						
32	32						
33	33						
34	34						
35	35						
36	36						
37	37						
38	38						
39	39						
40	40						
41	41						
42	42						
43	43						
44	44						
45	45						
46	46						
47	47						
48	48						
49	49						
50	50						
51	51						
52	52						
53	53						
54	54						
55	55						
56	56						
57	57						
58	58						
59	59						
60	60						
61	61						
62	62						
63	63						
64	64						
65	65						
66	66						
67	67						
68	68						
69	69						
70	70						
71	71						
72	72						
73	73						
74	74						
75	75						
76	76						
77	77						
78	78						
79	79						
80	80						
81	81						
82	82						
83	83						
84	84						
85	85						
86	86						
87	87						
88	88						
89	89						
90	90						
91	91						



Brighter 100

Index	Value	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466	467	468	469	470	471	472	473	474	475	476	477	478	479	480	481	482	483	484	485	486	487	488	489	490	491	492	493	494	495	496	497	498	499	500	501	502	503	504	505	506	507	508	509	510	511	512	513	514	515	516	517	518	519	520	521	522	523	524	525	526	527	528	529	530	531	532	533	534	535	536	537	538	539	540	541	542	543	544	545	546	547	548	549	550	551	552	553	554	555	556	557	558	559	560	561	562	563	564	565	566	567	568	569	570	571	572	573	574	575	576	577	578	579	580	581	582	583	584	585	586	587	588	589	590	591	592	593	594	595	596	597	598	599	600	601	602	603	604	605	606	607	608	609	610	611	612	613	614	615	616	617	618	619	620	621	622	623	624	625	626	627	628	629	630	631	632	633	634	635	636	637	638	639	640	641	642	643	644	645	646	647	648	649	650	651	652	653	654	655	656	657	658	659	660	661	662	663	664	665	666	667	668	669	670	671	672	673	674	675	676	677	678	679	680	681	682	683	684	685	686	687	688	689	690	691	692	693	694	695	696	697	698	699	700	701	702	703	704	705	706	707	708	709	710	711	712	713	714	715	716	717	718	719	720	721	722	723	724	725	726	727	728	729	730	731	732	733	734	735	736	737	738	739	740	741	742	743	744	745	746	747	748	749	750	751	752	753	754	755	756	757	758	759	760	761	762	763	764	765	766	767	768	769	770	771	772	773	774	775	776	777	778	779	780	781	782	783	784	785	786	787	788	789	790	791	792	793	794	795	796	797	798	799	800	801	802	803	804	805	806	807	808	809	810	811	812	813	814	815	816	817	818	819	820	821	822	823	824	825	826	827	828	829	830	831	832	833	834	835	836	837	838	839	840	841	842	843	844	845	846	847	848	849	850	851	852	853	854	855	856	857	858	859	860	861	862	863	864	865	866	867	868	869	870	871	872	873	874	875	876	877	878	879	880	881	882	883	884	885	886	887	888	889	890	891	892	893	894	895	896	897	898	899	900	901	902	903	904	905	906	907	908	909	910	911	912	913	914	915	916	917	918	919	920	921	922	923	924	925	926	927	928	929	930	931	932	933	934	935	936	937	938	939	940	941	942	943	944	945	946	947	948	949	950	951	952	953	954	955	956	957	958	959	960	961	962	963	964	965	966	967	968	969	970	971	972	973	974	975	976	977	978	979	980	981	982	983	984	985	986	987	988	989	990	991	992	993	994	995	996	997	998	999	1000
-------	-------	---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	------



Darker 50

Addr	Value	(7,199,399,399) hex	(7,199,399,399) hex	(7,199,399,399) hex	(7,199,399,399) hex	(7,199,399,399) hex
0x00000000	0					
0x00000001	0					
0x00000002	0					
0x00000003	0					
0x00000004	0					
0x00000005	0					
0x00000006	0					
0x00000007	0					
0x00000008	0					
0x00000009	0					
0x0000000A	0					
0x0000000B	0					
0x0000000C	0					
0x0000000D	0					
0x0000000E	0					
0x0000000F	0					
0x00000010	0					
0x00000011	0					
0x00000012	0					
0x00000013	0					
0x00000014	0					
0x00000015	0					
0x00000016	0					
0x00000017	0					
0x00000018	0					
0x00000019	0					
0x0000001A	0					
0x0000001B	0					
0x0000001C	0					
0x0000001D	0					
0x0000001E	0					
0x0000001F	0					
0x00000020	0					
0x00000021	0					
0x00000022	0					
0x00000023	0					
0x00000024	0					
0x00000025	0					
0x00000026	0					
0x00000027	0					
0x00000028	0					
0x00000029	0					
0x0000002A	0					
0x0000002B	0					
0x0000002C	0					
0x0000002D	0					
0x0000002E	0					
0x0000002F	0					
0x00000030	0					
0x00000031	0					
0x00000032	0					
0x00000033	0					
0x00000034	0					
0x00000035	0					
0x00000036	0					
0x00000037	0					
0x00000038	0					
0x00000039	0					
0x0000003A	0					
0x0000003B	0					
0x0000003C	0					
0x0000003D	0					
0x0000003E	0					
0x0000003F	0					
0x00000040	0					
0x00000041	0					
0x00000042	0					
0x00000043	0					
0x00000044	0					
0x00000045	0					
0x00000046	0					
0x00000047	0					
0x00000048	0					
0x00000049	0					
0x0000004A	0					
0x0000004B	0					
0x0000004C	0					
0x0000004D	0					
0x0000004E	0					
0x0000004F	0					
0x00000050	0					
0x00000051	0					
0x00000052	0					
0x00000053	0					
0x00000054	0					
0x00000055	0					
0x00000056	0					
0x00000057	0					
0x00000058	0					
0x00000059	0					
0x0000005A	0					
0x0000005B	0					
0x0000005C	0					
0x0000005D	0					
0x0000005E	0					
0x0000005F	0					
0x00000060	0					
0x00000061	0					
0x00000062	0					
0x00000063	0					
0x00000064	0					
0x00000065	0					
0x00000066	0					
0x00000067	0					
0x00000068	0					
0x00000069	0					
0x0000006A	0					
0x0000006B	0					
0x0000006C	0					
0x0000006D	0					
0x0000006E	0					
0x0000006F	0					
0x00000070	0					
0x00000071	0					
0x00000072	0					
0x00000073	0					
0x00000074	0					
0x00000075	0					
0x00000076	0					
0x00000077	0					
0x00000078	0					
0x00000079	0					
0x0000007A	0					
0x0000007B	0					
0x0000007C	0					
0x0000007D	0					
0x0000007E	0					
0x0000007F	0					
0x00000080	0					
0x00000081	0					
0x00000082	0					
0x00000083	0					
0x00000084	0					
0x00000085	0					
0x00000086	0					
0x00000087	0					
0x00000088	0					
0x00000089	0					
0x0000008A	0					
0x0000008B	0					
0x0000008C	0					
0x0000008D	0					
0x0000008E	0					
0x0000008F	0					
0x00000090	0					
0x00000091	0					
0x00000092	0					
0x00000093	0					
0x00000094	0					
0x00000095	0					
0x00000096	0					
0x00000097	0					
0x00000098	0					
0x00000099	0					
0x0000009A	0					
0x0000009B	0					
0x0000009C	0					
0x0000009D	0					
0x0000009E	0					
0x0000009F	0					
0x000000A0	0					
0x000000A1	0					
0x000000A2	0					
0x000000A3	0					
0x000000A4	0					
0x000000A5	0					
0x000000A6	0					
0x000000A7	0					
0x000000A8	0					
0x000000A9	0					
0x000000AA	0					
0x000000AB	0					
0x000000AC	0					
0x000000AD	0					
0x000000AE	0					
0x000000AF	0					
0x000000B0	0					
0x000000B1	0					
0x000000B2	0					
0x000000B3	0					
0x000000B4	0					
0x000000B5	0					
0x000000B6	0					
0x000000B7	0					
0x000000B8	0					
0x000000B9	0					
0x000000BA	0					
0x000000BB	0					
0x000000BC	0					
0x000000BD	0					
0x000000BE	0					
0x000000BF	0					
0x000000C0	0					
0x000000C1	0					
0x000000C2	0					
0x000000C3	0					
0x000000C4	0					
0x000000C5	0					
0x000000C6	0					
0x000000C7	0					
0x000000C8	0					
0x000000C9	0					
0x000000CA	0					
0x000000CB	0					
0x000000CC	0					
0x000000CD	0					
0x000000CE	0					
0x000000CF	0					
0x000000D0	0					
0x000000D1	0					
0x000000D2	0					
0x000000D3	0					
0x000000D4	0					
0x000000D5	0					
0x000000D6	0					
0x000000D7	0					
0x000000D8	0					
0x000000D9	0					
0x000000DA	0					
0x000000DB	0					
0x000000DC	0					
0x000000DD	0					
0x000000DE	0					
0x000000DF	0					
0x000000E0	0					
0x000000E1	0					
0x000000E2	0					
0x000000E3	0					
0x000000E4	0					
0x000000E5	0					
0x000000E6	0					
0x000000E7	0					
0x000000E8	0					
0x000000E9	0					
0x000000EA	0					
0x000000EB	0					
0x000000EC	0					
0x000000ED	0					
0x000000EE	0					
0x000000EF	0					
0x000000F0	0					
0x000000F1	0					
0x000000F2	0					
0x000000F3	0					
0x000000F4	0					
0x000000F5	0					
0x000000F6	0					
0x000000F7	0					
0x000000F8	0					
0x000000F9	0					
0x000000FA	0					
0x000000FB	0					
0x000000FC	0					
0x000000FD	0					
0x000000FE	0					
0x000000FF	0					



Darker 100

Addr	Value	(7,199,399,399) hex	(7,199,399,399) hex	(7,199,399,399) hex	(7,199,399,399) hex	(7,199,399,399) hex	(7,199,399,399) hex
0x00000000	0						
0x00000001	1						
0x00000002	2						
0x00000003	3						
0x00000004	4						
0x00000005	5						
0x00000006	6						
0x00000007	7						
0x00000008	8						
0x00000009	9						
0x0000000A	10						
0x0000000B	11						
0x0000000C	12						
0x0000000D	13						
0x0000000E	14						
0x0000000F	15						
0x00000010	16						
0x00000011	17						
0x00000012	18						
0x00000013	19						
0x00000014	20						
0x00000015	21						
0x00000016	22						
0x00000017	23						
0x00000018	24						
0x00000019	25						
0x0000001A	26						
0x0000001B	27						
0x0000001C	28						
0x0000001D	29						
0x0000001E	30						
0x0000001F	31						
0x00000020	32						
0x00000021	33						
0x00000022	34						
0x00000023	35						
0x00000024	36						
0x00000025	37						
0x00000026	38						
0x00000027	39						
0x00000028	40						
0x00000029	41						
0x0000002A	42						
0x0000002B	43						
0x0000002C	44						
0x0000002D	45						
0x0000002E	46						
0x0000002F	47						
0x00000030	48						
0x00000031	49						
0x00000032	50						
0x00000033	51						
0x00000034	52						
0x00000035	53						
0x00000036	54						
0x00000037	55						
0x00000038	56						
0x00000039	57						
0x0000003A	58						
0x0000003B	59						
0x0000003C	60						
0x0000003D	61						
0x0000003E	62						
0x0000003F	63						
0x00000040	64						
0x00000041	65						
0x00000042	66						
0x00000043	67						
0x00000044	68						
0x00000045	69						
0x00000046	70						
0x00000047	71						
0x00000048	72						
0x00000049	73						
0x0000004A	74						
0x0000004B	75						
0x0000004C	76						
0x0000004D	77						
0x0000004E	78						
0x0000004F	79						
0x00000050	80						
0x00000051	81						
0x00000052	82						
0x00000053	83						
0x00000054	84						
0x00000055	85						
0x00000056	86						
0x00000057	87						
0x00000058	88						
0x00000059	89						
0x0000005A	90						
0x0000005B	91						
0x0000005C	92						
0x0000005D	93						
0x0000005E	94						
0x0000005F	95						
0x00000060	96						
0x00000061	97						
0x00000062	98						
0x00000063	99						
0x00000064	100						
0x00000065	101						
0x00000066	102						
0x00000067	103						
0x00000068	104						
0x00000069	105						
0x0000006A	106						
0x0000006B	107						
0x0000006C	108						
0x0000006D	109						
0x0000006E	110						
0x0000006F	111						
0x00000070	112						
0x00000071	113						
0x00000072	114						
0x00000073	115						
0x00000074	116						
0x00000075	117						
0x00000076	118						
0x00000077	119						
0x00000078	120						
0x00000079	121						
0x0000007A	122						
0x0000007B	123						
0x0000007C	124						
0x0000007D	125						
0x0000007E	126						
0x0000007F	127						
0x00000080	128						
0x00000081	129						
0x00000082	130						
0x00000083	131						
0x00000084	132						
0x00000085	133						
0x00000086	134						
0x00000087	135						
0x00000088	136						
0x00000089	137						
0x0000008A	138						
0x0000008B	139						
0x0000008C	140						
0x0000008D	141						
0x0000008E	142						
0x0000008F	143						
0x00000090	144						
0x00000091	145						
0x00000092	146						
0x00000093	147						
0x00000094	148						
0x00000095	149						
0x00000096	150						
0x00000097	151						
0x00000098	152						
0x00000099	153						
0x0000009A	154						
0x0000009B	155						
0x0000009C	156						
0x0000009D	157						
0x0000009E	158						
0x0000009F	159						
0x000000A0	160						
0x000000A1	161						
0x000000A2	162						
0x000000A3	163						
0x000000A4	164						
0x000000A5	165						
0x000000A6	166						
0x000000A7	167						
0x000000A8	168						
0x000000A9	169						
0x000000AA	170						
0x000000AB	171						
0x000000AC	172						
0x000000AD	173						
0x000000AE	174						
0x000000AF	175						
0x000000B0	176						
0x000000B1	177						
0x000000B2	178						
0x000000B3	179						
0x000000B4	180						
0x000000B5	181						
0x000000B6	182						
0x000000B7	183						
0x000000B8	184						
0x000000B9	185						
0x000000BA	186						
0x000000BB	187						
0x000000BC	188						
0x000000BD	189						
0x000000BE	190						
0x000000BF	191						
0x000000C0	192						
0x000000C1	193						
0x000000C2	194						
0x000000C3	195						
0x000000C4	196						
0x000000C5	197						
0x000000C6	198						
0x000000C7	199						
0x000000C8	200						
0x000000C9	201						
0x000000CA	202						
0x000000CB	203						
0x000000CC	204						
0x000000CD	205						
0x000000CE	206						
0x000000CF	207						
0x000000D0	208						
0x000000D1	209						
0x000000D2	210						
0x000000D3	211						
0x000000D4	212						
0x000000D5	213						
0x000000D6	214						
0x000000D7	215						
0x000000D8	216						
0x000000D9	217						
0x000000DA	218						
0x000000DB	219						
0x000000DC	220						
0x000000DD	221						
0x000000DE	222						
0x000000DF	223						
0x000000E0	224						
0x000000E1	225						
0x000000E2	226						
0x000000E3	227						
0x000000E4	228						
0x000000E5	229						
0x000000E6	230						
0x000000E7	231						
0x000000E8	232						
0x000000E9	233						
0x000000EA	234						
0x000000EB	235						
0x000000EC	236						
0x000000ED	237						
0x000000EE	238						
0x000000EF	239						
0x000000F0	240						
0x000000F1	241						
0x000000F2	242						

읽어 Display Model로 전달한다. lcd_frame_buffer의 주요 입력 신호는 clk(1 bit)는 시스템 클럭, en(1 bit)는 메모리 활성화 신호, we(1 bit)는 쓰기 제어 신호, addr(W_WORD bit)는 AHB Bus를 통한 메모리 주소, din(24 bit)은 RGB888 형식의 입력 이미지 데이터이며, addr_dual_pixel(W_WORD bit)은 두 개의 Pixel을 동시에 읽기 위한 주소이다. 출력 신호는 dout(24 bit)으로 단일 Pixel의 RGB 데이터를 출력하며, dout_dual_pixel(48 bit)은 두 개의 RGB Pixel(24 bit × 2)을 동시에 출력하여 LCD Drive가 연속적으로 이미지를 표시할 수 있도록 한다.

Frame Buffer는 addr를 이용하여 AHB Bus의 읽기 및 쓰기 동작을 수행하며, LCD 출력 시에는 addr_dual_pixel을 이용하여 두 개의 Pixel을 동시에 읽는다. 읽어온 48 bit 데이터는 첫 번째 Pixel(R0, G0, B0)과 두 번째 Pixel(R1, G1, B1)으로 분리되어 Brightness Adjustment Module을 거친 후 Display Model로 전달된다. 이러한 구조를 통해 한 번의 메모리 접근으로 두 개의 Pixel을 처리하여 LCD 출력 효율을 향상시킬 수 있다.

Frame Buffer의 크기는 화면 해상도와 Pixel 크기에 의해 결정된다. 기본 설계는 768 × 512 해상도를 사용하며 Pixel 하나는 24 bit(RGB888)로 구성된다. 따라서 전체 Frame Buffer의 크기는 $768 \times 512 \times 24 \text{ bit} = 9,437,184 \text{ bit}$ (약 1,125 KB)이다. 본 실습에서는 시뮬레이션 시간을 단축하기 위해 top_system.v에서 256 × 256 해상도로 변경하여 실험을 수행하였으며, 이 경우 Frame Buffer의 크기는 1,572,864 bit(196,608 Byte, 약 192 KB)이다.

이번 실습을 통해 AHB Bus를 이용한 IP 통합 과정과 LCD Controller 기반의 이미지 출력 메커니즘을 이해할 수 있었으며, Frame Buffer를 이용한 이미지 저장 및 출력 방식과 Brightness 제어 기능의 동작 원리를 학습할 수 있었다.

Day10. LAB3 – Optimized Frame Buffer

본 실습에서는 LCD Drive의 AHB Slave Interface를 구현하고 이를 기존 SoC 시스템에 통합한 후, RCT(Reversible Color Transform)를 이용한 Compressed Frame Buffer를 구현하였다. 기존 시스템은 AHB Master(RISC-V Dummy)와 ALU, Multiplier, Memory로 구성되어 있었으며, LCD Drive를 하나의 AHB Slave IP로 추가하여 Bus Interconnect를 통해 접근할 수 있도록 설계하였다. 이를 위해 lcd_drive_if.v에서 LCD Drive의 AHB Slave Interface를 구현하고, top_system.v에서 LCD Drive를 Bus Interconnect에 연결하여 AHB Master가 LCD Drive의 제어 Register와 Frame Buffer에 접근할 수 있도록 하였다.

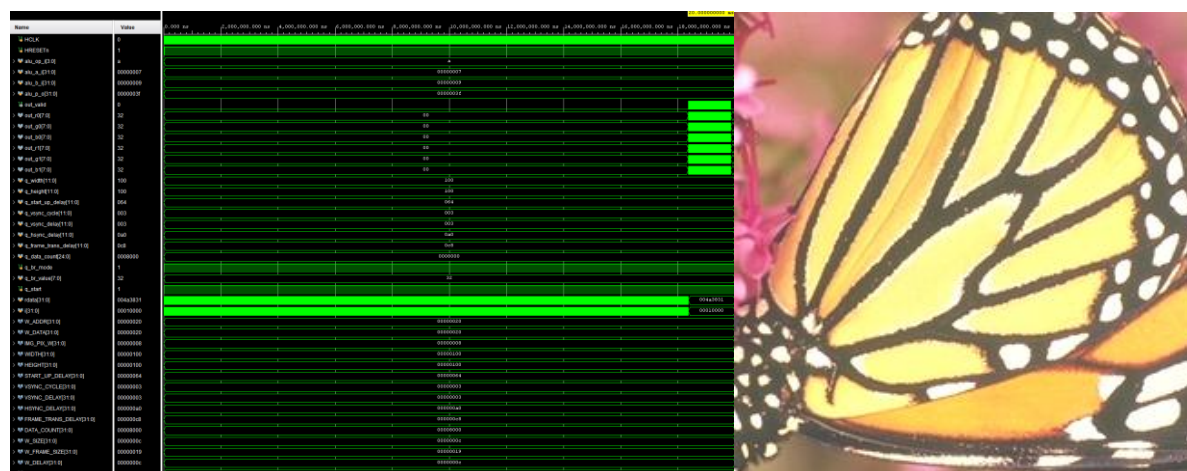
LCD Drive Interface는 AHB Slave Interface를 기반으로 구현되었으며, Address Phase에서는 전달되는 HADDR를 이용하여 접근할 Register를 선택하고 HWRITE 신호를 저장하여 Read/Write Transaction을 구분하도록 Address Decode Logic을 구성하였다. Data Phase에

서는 Width, Height, Start-up Delay, VSYNC Cycle, VSYNC Delay, HSYNC Delay, Frame Transition Delay, Data Count, Brightness Mode, Brightness Value 및 Start Register에 대해 읽기 및 쓰기 기능을 구현하였다. 또한 Register에 저장된 설정 값은 LCD Drive 내부 FSM의 동작 파라미터로 사용되어 화면 크기와 동기 신호 생성, 프레임 출력 및 밝기 조절 기능을 제어하도록 설계하였다.

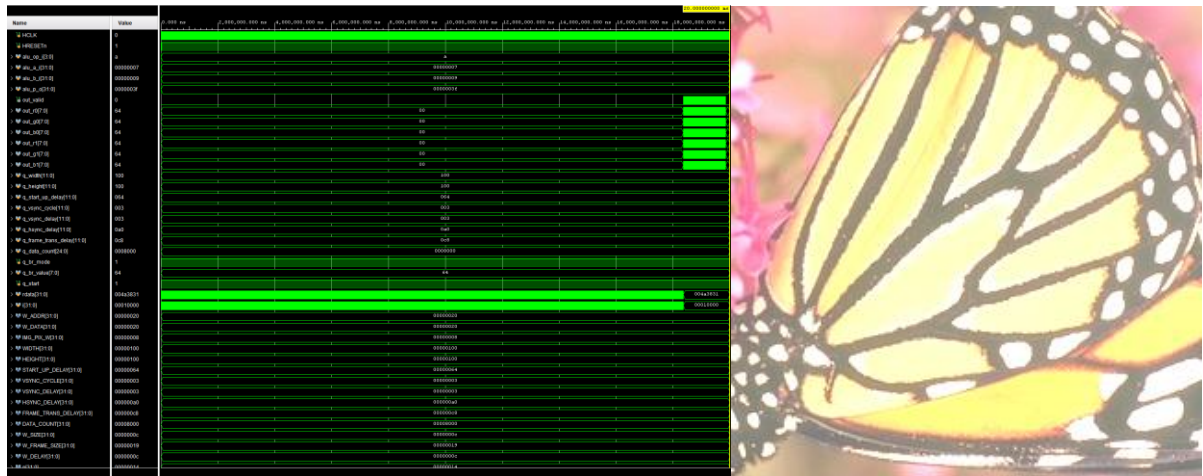
기존 Frame Buffer는 RGB888 형식의 이미지를 그대로 저장하였으나, 본 실습에서는 메모리 사용량을 감소시키기 위해 RCT(Reversible Color Transform)를 적용한 Compressed Frame Buffer를 구현하였다. 입력되는 두 개의 RGB Pixel은 lcd_forward_rct 모듈에서 각각 Y0, Y1과 공통 색차 성분인 Cb, Cr로 변환된 후 Frame Buffer에 저장된다. 이때 저장되는 데이터는 {Cr, Cb, Y1, Y0}의 형태이며 각 성분은 10 bit로 표현되어 총 40 bit의 데이터가 하나의 메모리 주소에 저장된다. LCD 출력 시에는 lcd_inverse_rct 모듈이 저장된 RCT 데이터를 다시 RGB 이미지로 복원하며, 복원된 이미지는 Brightness Adjustment Module을 거쳐 Display Model로 전달된다. 이를 통해 원본 이미지의 화질을 유지하면서도 Frame Buffer의 저장 공간을 줄일 수 있도록 하였다.

시뮬레이션 결과는 아래 그림들과 같다. (이미지의 순서대로 원본, Brighter 50, Brighter 100, Darker 50, Darker 100에 대한 Waveform 및 결과 bmp 파일이다.)

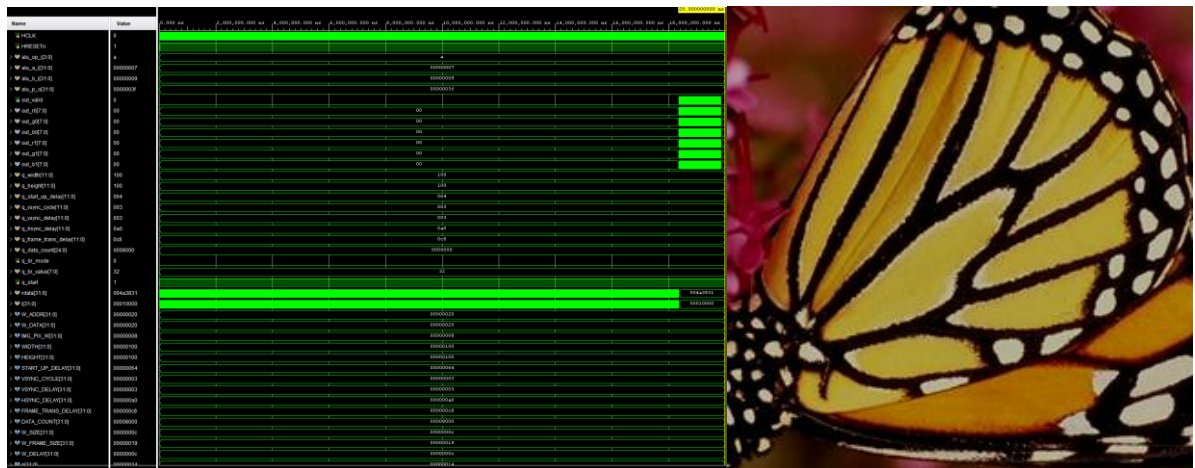
Brighter 50



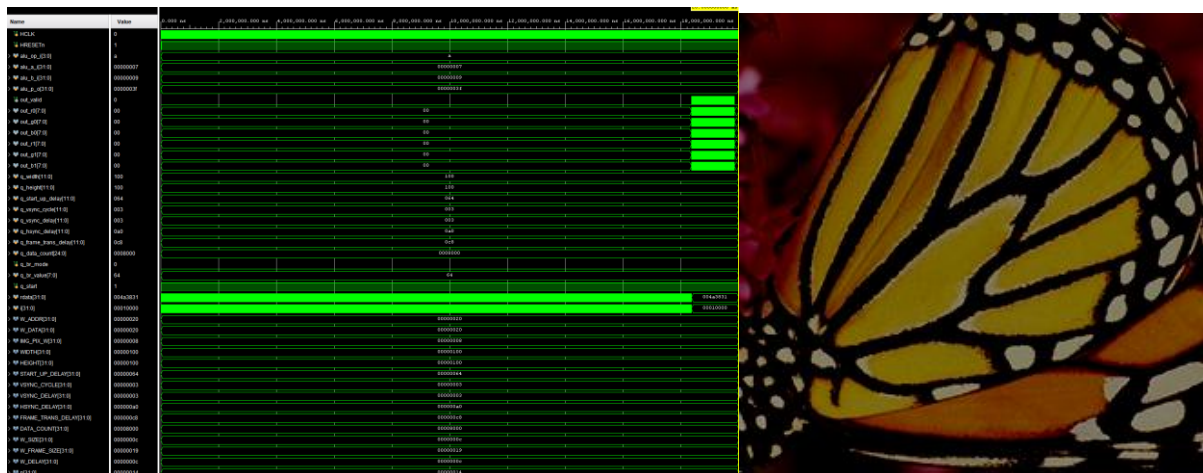
Brighter 100



Darker 50



Darker 100



시뮬레이션은 20 ms 동안 수행하였으며, 다양한 Brightness Mode를 적용하여 시스템의 정상 동작을 확인하였다. Waveform을 보면 AHB Master가 LCD Drive Register에 화면

크기, 동기 신호 지연 시간 및 데이터 개수 등의 초기 설정 값을 기록한 후 Start Register를 활성화하여 이미지 출력을 시작한다. 이후 LCD Drive 내부 FSM은 IDLE → VSYNC → HSYNC → DATA 순으로 상태를 전이하며, Row Counter와 Column Counter를 증가시켜 화면 전체를 순차적으로 스캔한다. DATA 상태에서는 Frame Buffer로부터 압축된 RCT 데이터를 읽어온 후 Inverse RCT를 통해 RGB 데이터로 복원하고, Brightness Adjustment Module을 거쳐 Display Model로 전달하는 것을 확인하였다. 또한 q_br_mode와 q_br_value를 변경함으로써 출력 이미지의 밝기가 정상적으로 증가하거나 감소하는 것을 확인하였으며, 생성된 BMP 파일에서도 이미지의 구조는 그대로 유지되면서 밝기만 정상적으로 변경되는 것을 확인하였다.

Compressed Frame Buffer인 lcd_frame_buffer_opt는 Block RAM을 이용하여 RCT 압축 데이터를 저장하도록 구현하였다. 입력 신호는 clk(1 bit), en(1 bit), we(1 bit), addr(18 bit), din(40 bit), addr_dual_pixel(18 bit)로 구성된다. clk는 시스템 클럭, en은 메모리 활성화 신호, we는 쓰기 제어 신호이며, addr는 일반적인 읽기 및 쓰기를 위한 주소이다. din은 두 개의 Pixel을 RCT로 변환한 압축 데이터({Cr, Cb, Y1, Y0})를 저장하며, addr_dual_pixel은 LCD 출력 과정에서 두 Pixel 데이터를 동시에 읽기 위한 주소로 사용된다. 출력 신호는 dout(40 bit)과 dout_dual_pixel(40 bit)으로 구성되며, dout은 일반적인 읽기 데이터를 출력하고, dout_dual_pixel은 LCD Drive가 화면을 출력하기 위해 두 Pixel의 압축 데이터를 동시에 읽어오는 데 사용된다.

Frame Buffer의 크기는 저장되는 데이터의 크기와 메모리 개수에 의해 결정된다. LAB2 (Problem 5)에서는 RGB888 형식의 Pixel 하나를 24 bit로 저장하였으며, 총 768×512 개의 Pixel을 저장하므로 Frame Buffer의 크기는 $768 \times 512 \times 24 \text{ bit} = 9,437,184 \text{ bit}$ (1,179,648 Byte, 약 1,152 KB)이다. 반면 Problem 6에서는 두 개의 Pixel을 하나의 메모리 주소에 저장하며, 각 주소에는 Y0, Y1, Cb, Cr의 총 40 bit 데이터가 저장된다. 따라서 메모리 개수는 $768 \times 512 / 2 = 196,608$ 개이며, 전체 Frame Buffer의 크기는 $196,608 \times 40 \text{ bit} = 7,864,320 \text{ bit}$ (983,040 Byte, 약 960 KB)가 된다. 결과적으로 기존 Frame Buffer 대비 약 192 KB의 메모리를 절약할 수 있으며, 약 16.7%의 메모리 사용량 감소 효과를 얻을 수 있었다.

표로 정리하면 아래와 같다.

항목	LAB2	LAB3
저장 방식	RGB888	RCT (Y0, Y1, Cb, Cr)
저장 단위	1 Pixel (24 bit)	2 Pixels (40 bit)
메모리 개수	393,216	196,608
Frame Buffer 크기	9,437,184 bit (1,179,648 Byte, 약 1,152 KB)	7,864,320 bit (983,040 Byte, 약 960 KB)

이번 실습을 통해 AHB Bus 기반의 IP 통합 과정과 LCD Controller의 이미지 출력 구조를 이해할 수 있었으며, RCT 기반의 압축 기법을 이용하여 Frame Buffer의 메모리 사용량을 감소시키는 방법을 학습하였다. 또한 Forward RCT와 Inverse RCT를 이용하여 압축된 이미지를 다시 RGB 이미지로 복원할 수 있음을 확인하였으며, 메모리 사용량을 줄이면서도 원본 이미지의 출력 품질을 유지할 수 있음을 확인하였다.

Day11. LAB1 – Convolutional Layer

본 실습에서는 Matlab을 이용하여 CNN(Convolutional Neural Network)의 Convolution Layer를 직접 구현하고, 각 Layer의 Feature Map을 생성하는 과정을 수행하였다. 또한 각 Convolution Layer에서 필요한 곱셈(Multiplication) 연산 횟수를 계산하고, 전체 네트워크의 연산량을 분석하였다.

먼저 convol2.m의 빈 부분을 완성하여 Convolution 연산을 수행하도록 구현하였다. 구현한 함수는 입력 Feature Map과 Convolution Kernel을 입력받아 Stride와 Padding을 고려한 Convolution 연산을 수행하고, 출력 Feature Map을 생성한다.

Convolution 연산은 출력 Feature Map의 모든 위치에 대해 다음 과정을 반복한다.

1. 현재 위치에 대응하는 입력 Feature Map 영역(3×3)을 선택한다.
2. 대응되는 Filter Weight와 원소별 곱셈(Element-wise Multiplication)을 수행한다.
3. 3×3 영역의 결과를 모두 더한다.
4. 입력 채널이 여러 개인 경우 모든 입력 채널의 결과를 누적하여 하나의 출력 Pixel을 생성한다.

test_SR.m에서는 사전 학습된 ESPCN Weight를 읽어와 총 3개의 Convolution Layer를 순차적으로 수행하였다.

Layer 1

- Convolution
- Bias Addition
- ReLU Activation
- 16개의 Feature Map 출력

Layer 2

- Convolution
- Bias Addition

- ReLU Activation
- 16개의 Feature Map 출력

Layer 3

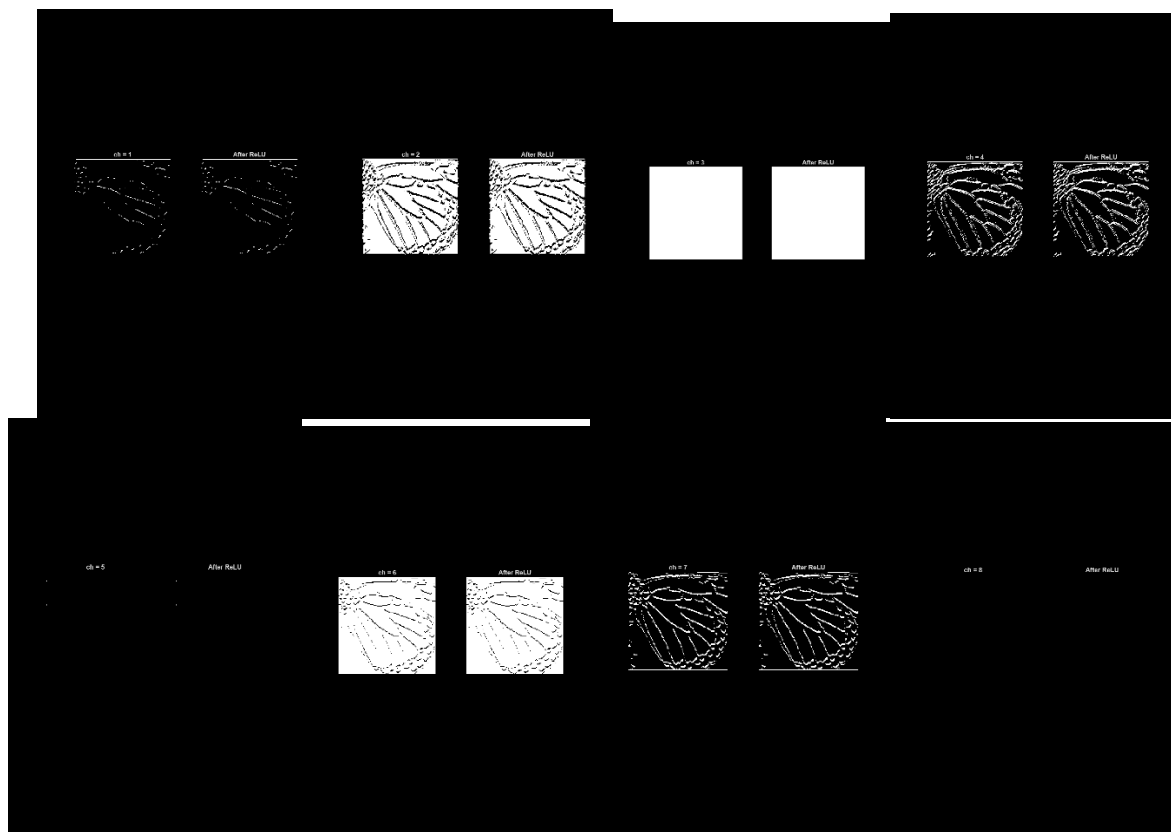
- Convolution
- Bias Addition
- 최종 출력 Layer이므로 ReLU는 적용하지 않음
- 4개의 Feature Map 출력

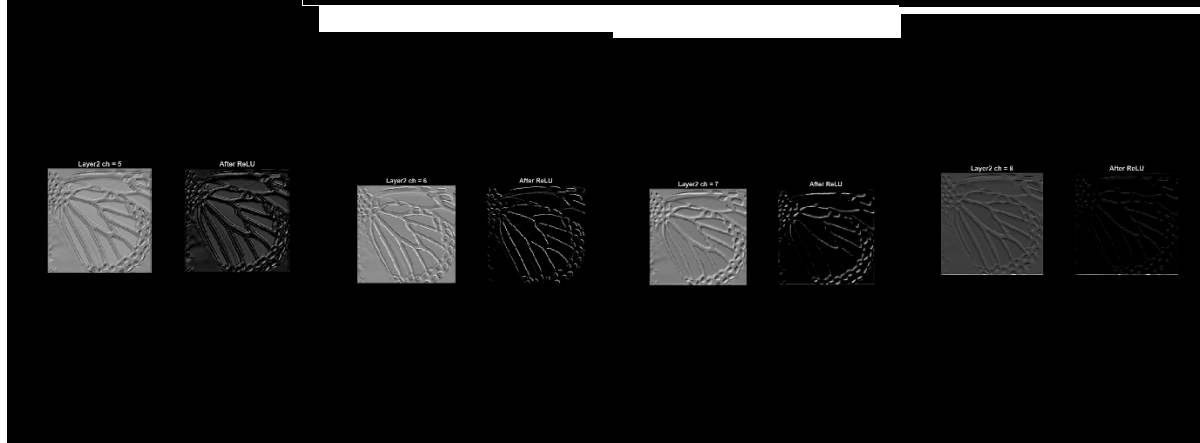
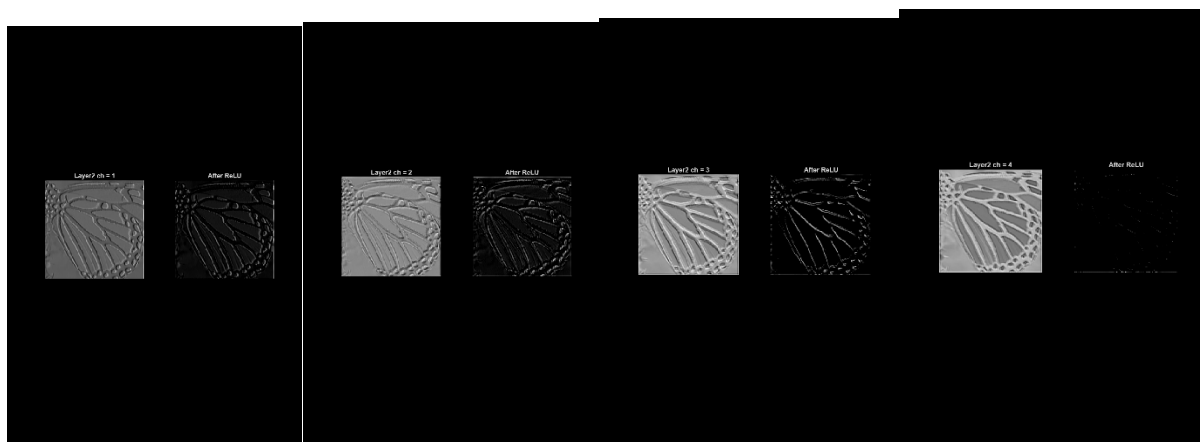
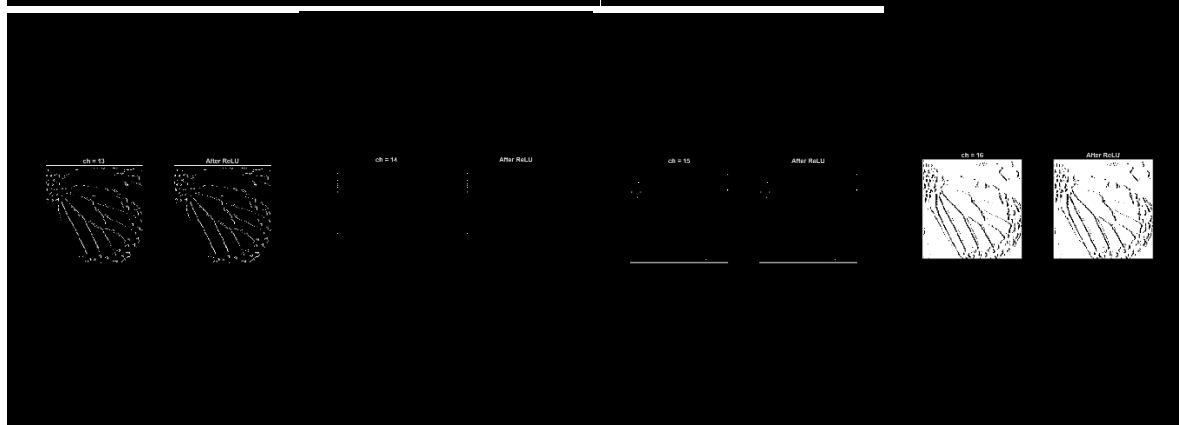
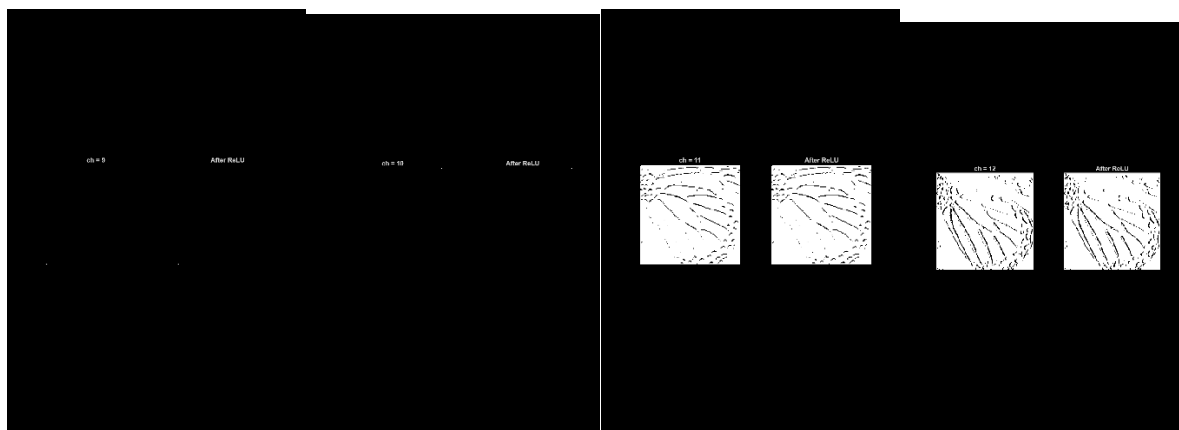
총 출력되는 Feature Map의 개수는 다음과 같다.

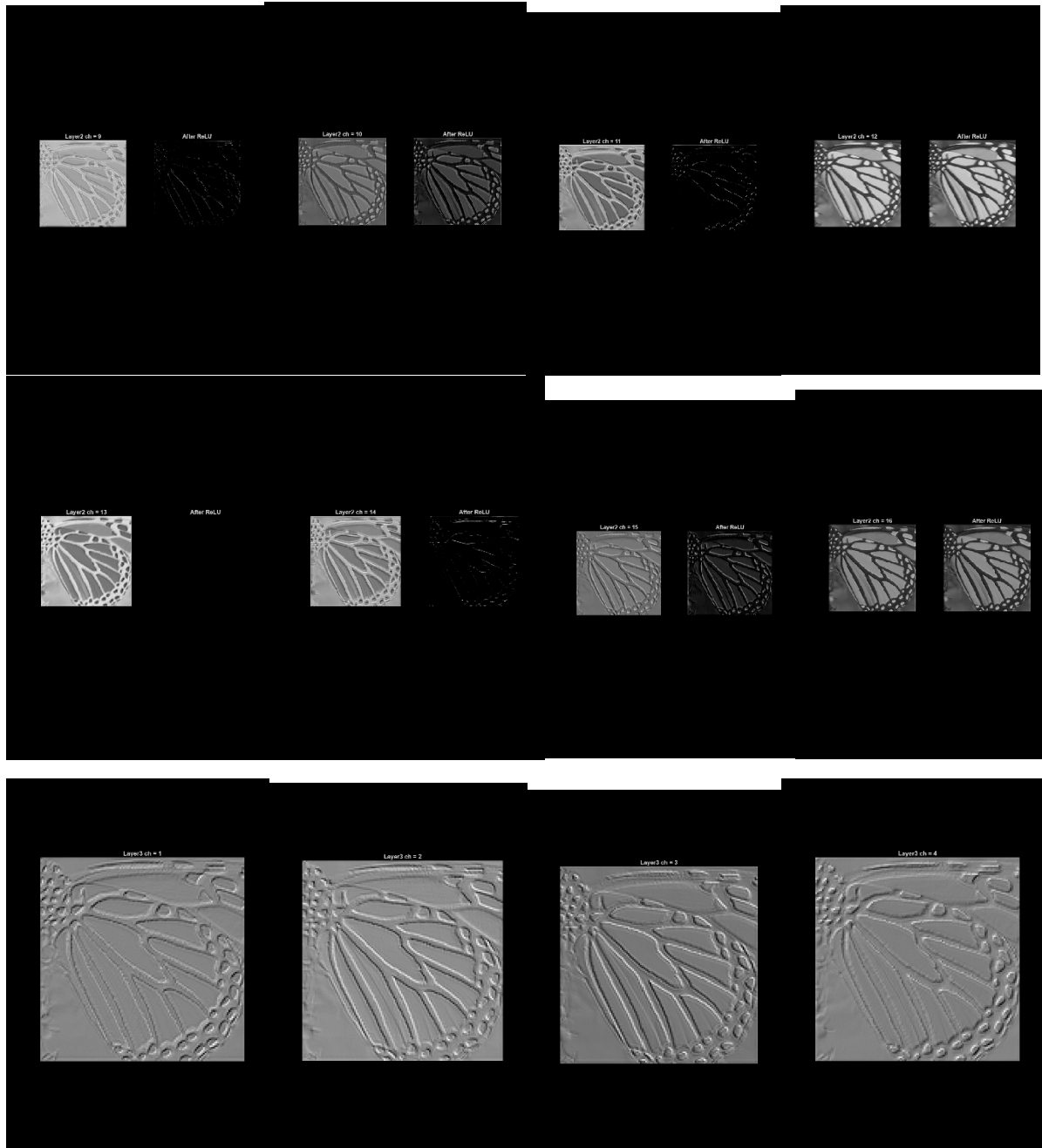
Layer	Number of Feature Map
Layer 1	16
Layer 2	16
Layer 3	4
Total	36

생성된 Feature Map은 각각 BMP 이미지로 저장하여 각 Layer에서 특징 정보가 어떻게 변화하는지 확인하였다.

생성된 Feature Map들은 아래 사진과 같다.







또한, 출력 Pixel 하나를 계산하는 데 필요한 곱셈 횟수(num_ops)는 다음과 같이 계산된다.

$$num_ops = FilterHeight \times FilterWidth \times InputChannels$$

이에 따른 각 Layer의 연산량은 다음과 같다.

Layer	Filter Size	Input Channels	Output Channels	Input	Output	num_ops
1	3×3	1	16	128×128×1	128×128×16	9
2	3×3	16	16	128×128×16	128×128×16	144

3	3×3	16	4	128×128×16	128×128×4	144
---	-----	----	---	------------	-----------	-----

또한, 전체 연산 횟수는

$$OutputWidth \times OutputHeight \times OutputChannels \times num_ops$$

으로 계산된다.

Layer 1

$$128 \times 128 \times 16 \times 9 = 2,359,296$$

Layer 2

$$128 \times 128 \times 16 \times 144 = 37,748,736$$

Layer 3

$$128 \times 128 \times 4 \times 144 = 9,437,184$$

따라서 전체 곱셈 연산 횟수는

$$2,359,296 + 37,748,736 + 9,437,184 = 49,545,216$$

이다.

Day11. LAB2 – Quantization

본 실습에서는 CNN 기반 Super-Resolution 모델의 하드웨어 구현을 위해 Weight와 Activation에 대한 Quantization을 적용하였다. Floating-point 기반 연산은 높은 정확도를 제공하지만 하드웨어에서는 많은 메모리와 연산 자원을 요구하므로, 이를 Fixed-point 기반의 정수 연산으로 변환하여 연산량과 메모리 사용량을 감소시키는 것이 목적이다. 본 실습에서는 Problem 7에서 구현한 convol2.m을 그대로 이용하고, Weight/Bias Quantization(uniform_quantize.m)과 Activation Quantization(hwu_relu_quantize.m, hwu_linear_quantize.m)을 구현하여 세 개의 Convolution Layer에 적용하였다.

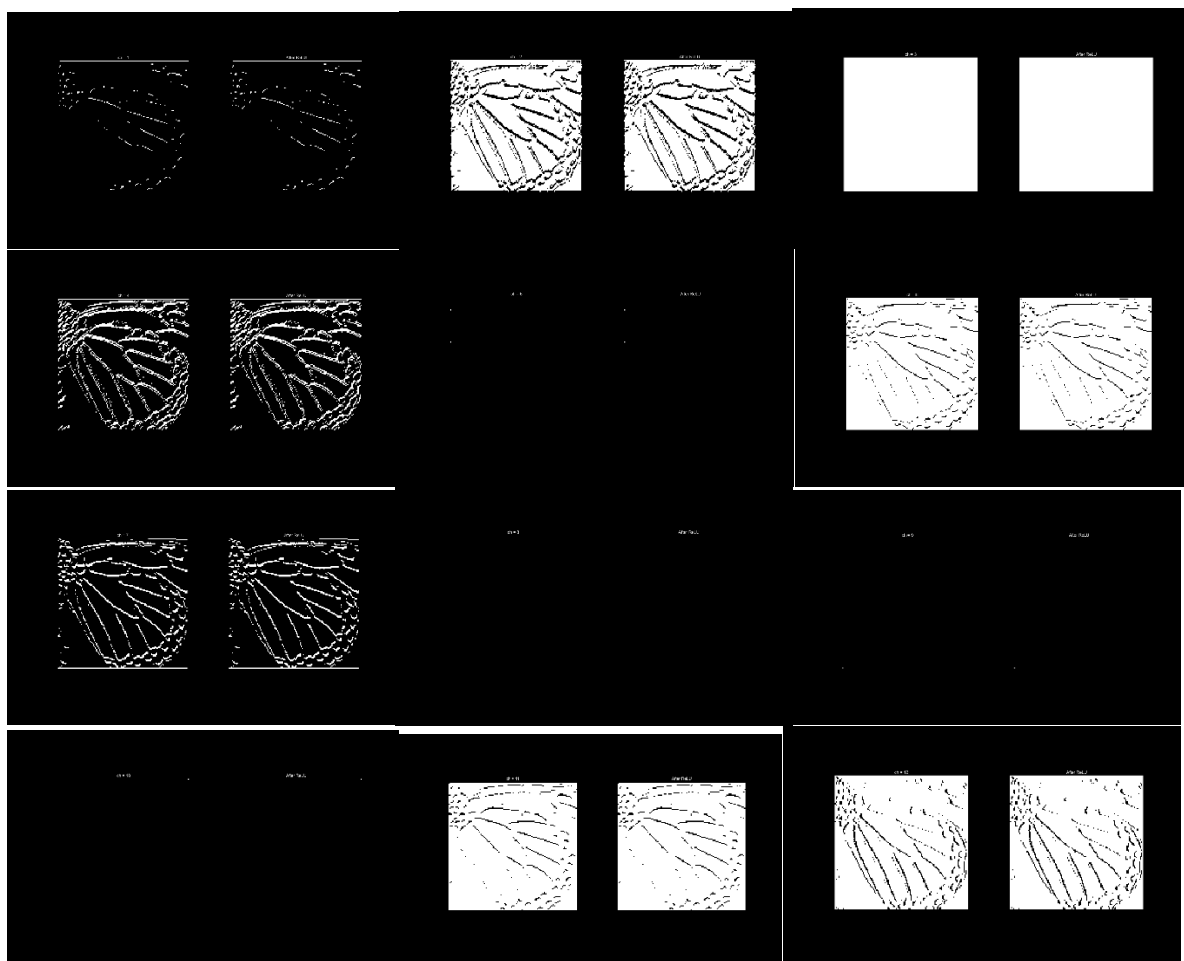
Weight와 Bias는 uniform_quantize.m에서 균일 양자화(Uniform Quantization)를 수행하였다. 입력된 Weight를 양자화 간격(step)으로 나눈 후 반올림(round)을 수행하여 정수 값으로 변환하고, 지정된 bit 수에 맞게 Saturation을 적용하였다. 이후 다시 step을 곱하여 Fixed-point 형태의 Weight를 생성하였다. 이를 통해 Floating-point Weight를 하드웨어에서 사용할 수 있는 정수 형태로 변환하였다.

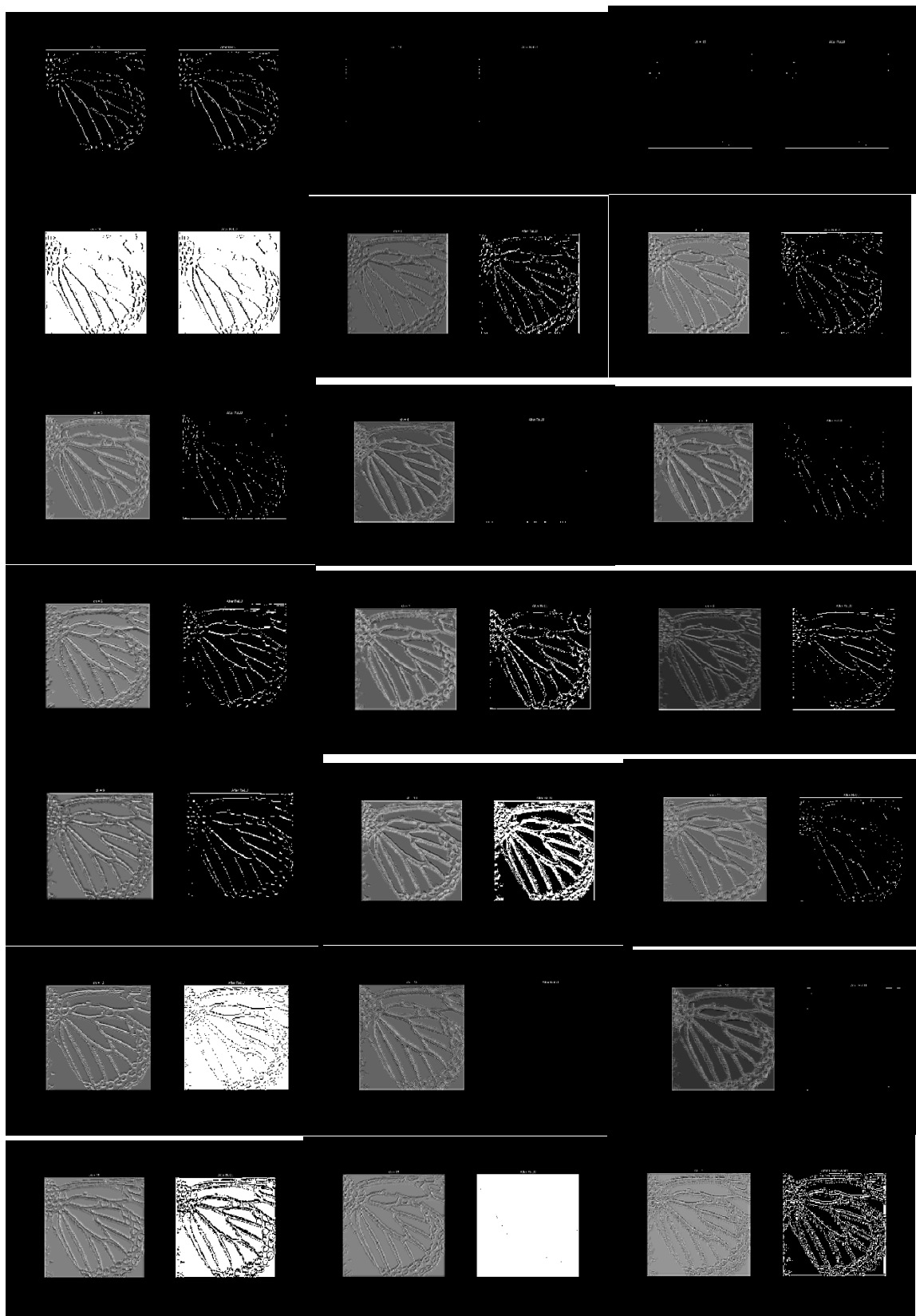
Activation Quantization은 각 Layer의 활성화 함수에 따라 서로 다른 방식으로 구현하였다. 첫 번째와 두 번째 Convolution Layer는 ReLU 활성화 함수를 사용하므로 hwu_relu_quantize.m을 구현하였다. 입력 Feature Map에서 음수 값은 모두 0으로 변경하

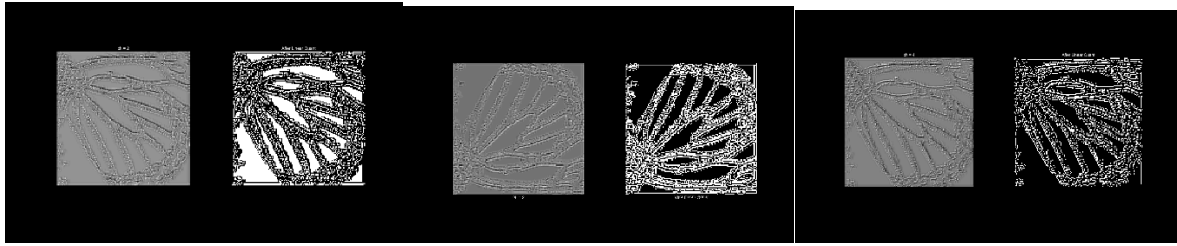
고, 양수 값은 step과 bias shift를 이용하여 양자화를 수행하였다. 양자화된 값은 지정된 bit 범위를 초과하지 않도록 Saturation을 적용하여 저장하였다. 마지막 세 번째 Layer는 출력층이므로 ReLU를 사용하지 않고 Linear Activation을 적용하였다. 따라서 hwu_linear_quantize.m에서는 입력 값을 step으로 나누어 양자화를 수행한 후 Signed Fixed-point 범위 내에서 Saturation을 적용하고 다시 Dequantization하여 출력 Feature Map을 생성하도록 구현하였다.

Quantization이 적용된 전체 CNN은 test_simSR_quant.m에서 수행하였다. 먼저 세 개의 Layer에 대해 Weight와 Bias를 각각 Quantization한 후, Layer 1과 Layer 2에서는 Convolution → Bias Addition → ReLU Quantization 순으로 계산을 수행하였다. 마지막 Layer에서는 Convolution → Bias Addition → Linear Quantization을 수행하여 최종 출력 Feature Map을 생성하였다. 각 Layer의 출력 Feature Map은 Channel별로 저장하고 화면에 출력하여 양자화 결과를 확인하였다.

시뮬레이션 결과는 아래 그림들과 같다.







출력된 Feature Map을 확인한 결과, Quantization을 적용한 이후에도 Feature Map의 전체적인 구조와 에지 정보는 대부분 유지되는 것을 확인하였다. Layer 1에서는 입력 이미지의 저 수준 특징이 추출되며, Layer 2에서는 더욱 복잡한 특징이 생성되었다. 마지막 Layer에서는 Super-Resolution을 위한 출력 Feature Map이 생성되었으며, Floating-point 결과와 비교했을 때 미세한 밝기 차이는 존재하지만 전체적인 이미지 구조는 거의 동일하게 유지되었다.

또한, Quantization 과정에서는 일부 작은 값들이 동일한 정수 값으로 변환되고, 표현 가능한 범위를 초과하는 값들은 Saturation에 의해 최대 또는 최소값으로 제한되는 것을 확인하였다. 이로 인해 약간의 양자화 오차(Quantization Error)가 발생하지만, 대부분의 Feature Map에서는 시각적으로 큰 차이를 보이지 않았다. 이는 Fixed-point 기반 구현에서도 CNN의 주요 특징 정보를 유지할 수 있음을 의미한다.

이번 실습을 통해 CNN을 하드웨어에서 구현하기 위한 Fixed-point Quantization 기법을 이해할 수 있었으며, Weight와 Activation을 정수화하여 메모리 사용량과 연산 복잡도를 감소시키는 과정을 학습하였다. 또한 ReLU와 Linear Activation의 양자화 방식 차이를 이해하고, Quantization 이후에도 CNN Feature Map이 정상적으로 생성되는 것을 확인하였다. 이를 통해 FPGA 및 ASIC 기반 CNN 가속기에서 Quantization이 필수적인 최적화 기법임을 확인할 수 있었다.

Day11. LAB3 – Block memory

본 실습에서는 CNN에서 사용하는 Filter Weight를 저장하기 위한 Dual Port Block RAM을 구현하고 Testbench를 작성하여 정상적으로 동작하는지 확인하였다. 기존 Single Port RAM과 달리 Dual Port Block RAM은 하나의 메모리에 대해 두 개의 독립적인 접근 포트를 제공한다. Port A는 쓰기(Write)와 읽기(Read)를 모두 수행할 수 있으며, Port B는 읽기(Read) 전용으로 동작하도록 설계하였다. 이러한 구조를 이용하면 Weight를 저장하는 동안에도 다른 포트에서 동시에 Weight를 읽을 수 있으므로 CNN Accelerator에서 높은 데이터 처리 효율을 얻을 수 있다.

Dual Port Block RAM은 128 bit 데이터를 하나의 주소에 저장하도록 구성하였다. 하나의 주소에는 16개의 8-bit Weight가 저장되며, CNN의 3×3 Filter Weight들을 순차적으로

저장한다. Testbench에서는 simSR.hex 파일에 저장된 Layer 1의 Convolution Filter를 Block RAM으로부터 읽어오도록 구현하였다. Read 동작을 위해 Read Enable(enb)를 활성화한 후 Address(addrb)를 순차적으로 증가시키며 각 주소에 저장된 Weight를 읽도록 구성하였다. 읽어온 128 bit 데이터는 각각 8 bit씩 분리하여 weight_store 배열에 저장하고, 이를 Unsigned에서 Signed 값으로 변환하여 weight 배열에 저장하도록 구현하였다.

시뮬레이션 결과는 아래 그림과 같다.



Waveform을 보면 Read Enable(enb)이 활성화될 때마다 Address(addrb)가 0부터 15까지 순차적으로 증가하는 것을 확인할 수 있다. 각 주소에서 Dual Port Block RAM은 128 bit 데이터(dob)를 정상적으로 출력하며, 출력된 데이터는 16개의 8 bit Weight로 분리되어 weight_store 배열에 저장된다. 이후 각 Weight는 Signed 형식으로 변환되어 weight 배열에 저장되는 것을 확인하였다.

또한 Waveform에서 dob의 값이 주소가 변경될 때마다 새로운 Weight 데이터로 갱신되며, weight_store와 weight 역시 동일한 시점에서 함께 변경되는 것을 확인하였다. weight_store에는 Hex 형태의 원본 Weight 값이 저장되고, weight에는 CNN 연산에서 사용할 Signed Weight가 저장되어 이후 Convolution 연산에 바로 사용할 수 있도록 구성하였다.

이번 실습을 통해 Dual Port Block RAM의 동작 원리와 CNN Accelerator에서 Weight

Memory가 동작하는 과정을 이해할 수 있었다. Dual Port 구조를 사용함으로써 Weight를 저장하는 과정과 읽는 과정을 독립적으로 수행할 수 있으며, 하나의 메모리 접근으로 16개의 Weight를 동시에 읽어올 수 있어 CNN 연산의 병렬성과 메모리 대역폭을 크게 향상시킬 수 있음을 확인하였다. 또한 Testbench를 이용하여 Layer 1의 Filter Weight가 simSR.hex 파일로부터 정상적으로 로드되고 Block RAM을 통해 순차적으로 출력되는 것을 검증하였다.

Day11. LAB4 – SRNPU Reference Software

본 실습에서는 앞서 실습한 Convolution Layer와 Weight, Activation Quantization을 재사용하여 SRNPU(Super Resolution Neural Processing Unit)를 구현하였다. SRNPU는 CNN 기반의 Super Resolution을 수행하는 하드웨어 구조로, Weight Buffer에 저장된 Filter를 이용하여 Convolution 연산을 수행하고 Bias를 더한 후 Activation Function을 적용하여 Feature Map을 생성한다. 이후 마지막 Layer에서는 Pixel Shuffle 과정을 수행하여 저해상도 이미지를 고해상도 이미지로 복원하도록 설계하였다.

SRNPU의 각 Convolution Layer는 Weight Buffer에서 Filter Weight를 읽어오는 과정으로 시작된다. 입력 Feature Map과 Weight를 이용하여 3×3 Convolution 연산을 수행한 후 각 Output Channel마다 Bias를 더하도록 구현하였다. Activation Function은 첫 번째와 두 번째 Layer에서는 ReLU Activation을 적용하여 음수 값을 제거하였으며, 마지막 Layer에서는 이미지의 출력값을 유지하기 위해 Linear Activation을 사용하였다. 또한 각 Layer에서 생성된 Output Feature Map은 다음 Layer의 입력으로 전달되며, 마지막 Layer의 출력은 Pixel Shuffle을 통해 최종 Super Resolution 이미지를 생성하도록 한다.

추가적으로 SRNPU에서는 Fixed-point 기반의 CNN 연산을 수행하기 위해 Problem 8에서 구현한 Quantization을 적용하였다. Weight와 Bias는 Uniform Quantization을 통해 정수 형태로 변환하였으며, Activation 역시 Quantization을 수행하여 Floating-point 대신 Fixed-point 환경에서 연산이 이루어지도록 한다.

또한 Fixed-point 연산에서는 Layer마다 서로 다른 Scale을 갖는 Weight, Bias 및 Activation을 동일한 Scale로 맞추기 위해 `bias_shift`와 `act_shift`를 사용하였다. Matlab 코드에서는 다음과 같이 정의된다.

1. `bias_shift = bit_shift{i};`
2. `act_shift = biases_fshift - act_fbit;`

`bias_shift`는 Bias를 Convolution 결과와 동일한 Fixed-point Scale로 변환하기 위한 Shift 값이다. Weight와 입력 Activation의 곱셈 결과는 기존 Bias와 서로 다른 Scale을 가지므

로, Bias를 적절히 Shift하여 동일한 Scale에서 덧셈이 수행되도록 한다. 이를 통해 서로 다른 Scale의 데이터를 더하는 오류를 방지하고 정확한 Fixed-point Convolution 연산을 수행할 수 있다.

반면 act_shift는 Bias Addition 이후 생성된 출력값을 다음 Layer의 Activation Scale에 맞추기 위한 Shift 값이다. Convolution 결과는 매우 큰 정수값으로 계산되므로 이를 다음 Layer에서 사용할 수 있는 Activation Bit Width 범위로 다시 변환해야 한다. act_shift는 이러한 Scale 조정을 수행하여 Layer 간 Activation Scale을 일정하게 유지하고 Overflow를 방지하며 안정적인 Fixed-point 연산이 가능하도록 한다. 즉, bias_shift는 Bias와 Convolution 결과의 Scale을 맞추기 위한 값이며, act_shift는 Layer 간 Activation의 Scale을 유지하기 위한 값이다. 두 Shift 값은 Quantization 환경에서 CNN 추론의 정확도를 유지하기 위해 반드시 필요한 파라미터이다.

시뮬레이션 결과는 아래 그림과 같다.



Command Window를 보면 SRNPU가 총 3개의 Convolution Layer를 순차적으로 수행한 후 Super Resolution 이미지를 생성하는 것을 확인할 수 있다. 각 Layer에서는 Weight Buffer에서 Filter를 읽어와 Convolution 연산을 수행하고 Bias Addition과 Activation을 적용한 뒤 다음 Layer로 Feature Map을 전달하였다. 또한 각 Layer에서 사용된 Scale과

bias_shift, act_shift 값이 함께 출력되어 Quantization 환경에서 Fixed-point 연산이 정상적으로 수행되고 있음을 확인하였다.

Feature Map을 확인한 결과 Layer 1에서는 입력 이미지의 Edge 및 저수준 특징이 추출되었으며, Layer 2에서는 보다 복잡한 Texture와 특징 정보가 생성되었다. 마지막 Layer에서는 최종 복원에 필요한 Feature Map이 생성되었으며, Pixel Shuffle을 수행하여 고해상도 이미지가 정상적으로 복원되는 것을 확인하였다. 이를 통해 각 Layer의 Convolution과 Activation이 정상적으로 수행되고 있음을 확인하였다.

또한 Command Window에서는 Bicubic Interpolation과 SRNPU를 이용한 Super Resolution 결과의 PSNR이 함께 출력되었다. 실험 결과 Bicubic Interpolation의 PSNR은 27.43 dB, SRNPU Reconstruction의 PSNR은 30.97 dB로 측정되었다. 이는 SRNPU를 이용한 CNN 기반 Super Resolution이 기존 Bicubic 보간법보다 약 3.54 dB 높은 PSNR을 나타내어 보다 정확한 이미지 복원이 가능함을 의미한다. 또한 Weight와 Activation에 Quantization을 적용한 Fixed-point 환경에서도 높은 복원 성능을 유지하는 것을 확인할 수 있었다.

이번 실습을 통해 CNN 기반 Super Resolution의 전체 추론 과정과 SRNPU의 내부 동작 원리를 이해할 수 있었다. Weight Buffer를 이용한 Filter 관리, Convolution 연산, Bias Addition, Activation 및 Quantization 과정이 하나의 CNN Accelerator에서 어떻게 수행되는지 확인하였으며, 특히 bias_shift와 act_shift를 이용하여 Layer 간 Fixed-point Scale을 유지하는 방법을 학습하였다. 또한 Fixed-point 기반의 Quantization을 적용하면서도 Bicubic Interpolation보다 우수한 이미지 복원 성능을 유지함을 확인하였으며, 이를 통해 실제 하드웨어 구현을 위한 CNN Accelerator의 동작 원리를 이해할 수 있었다.